ABAP INTERVIEW PREPARATION

Visual interview atlas

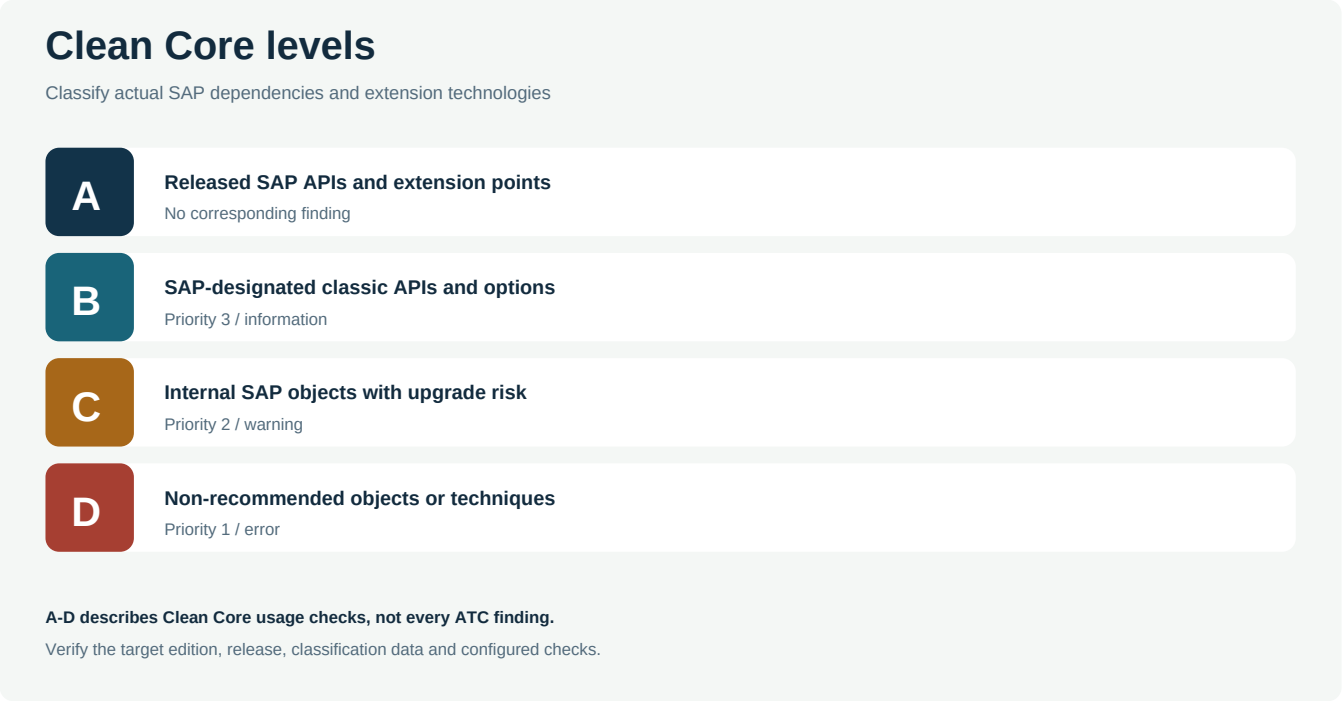
Original illustrations, data examples, comparison plates and concept maps

329 questions across 24 study segments. Reviewed 11 October 2026.

Use the short answers for oral rehearsal. The examples and pitfalls prepare you for follow-up questions. Choose the target SAP edition and release before applying an implementation detail.

Read online in GitHub through README.md, or download the repository and open START_HERE.html for offline recall cards. The PDF is a portable reading and printing edition.

Clean Core levels A-D



Read each level as a classification of the actual SAP dependency or extension technology.

Explain it: Which configured Clean Core finding priority corresponds to levels B, C and D?

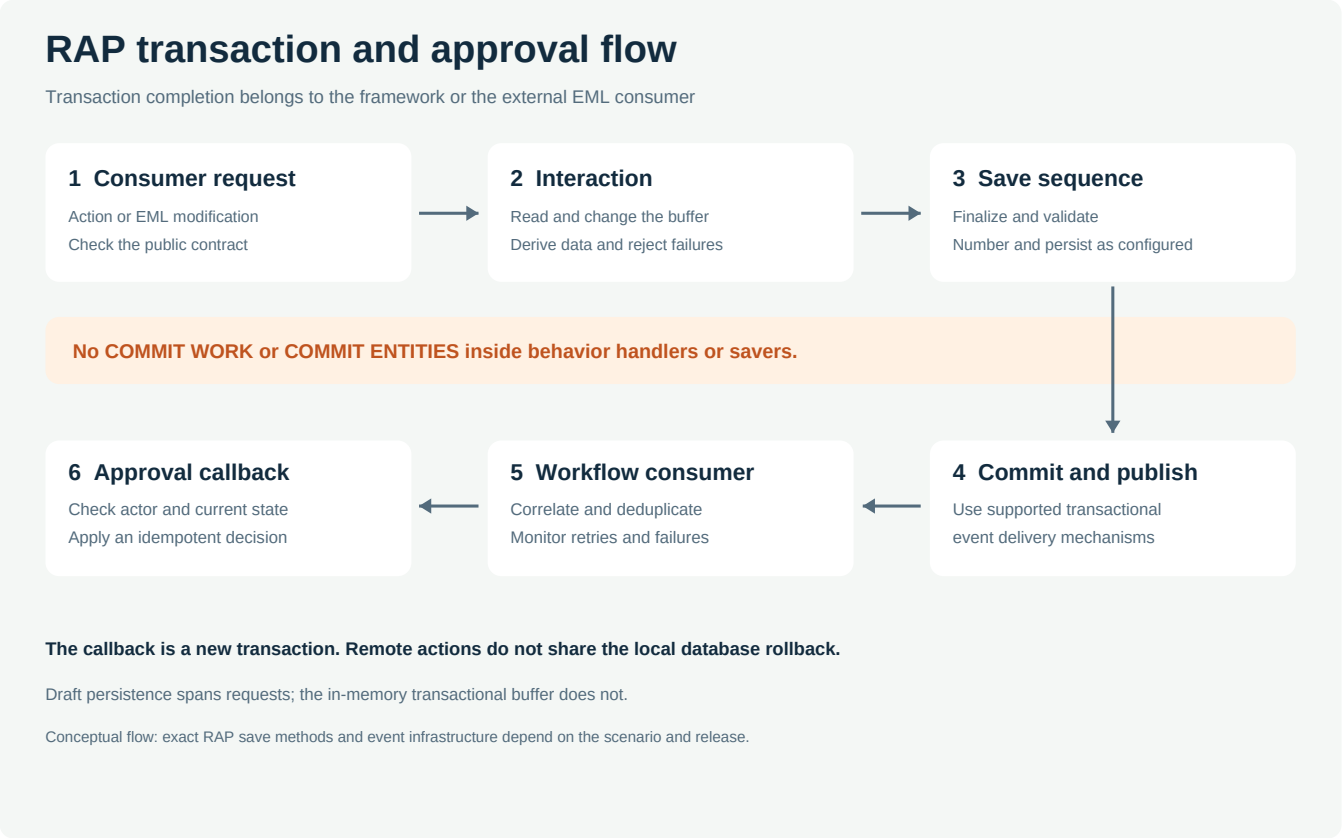
Limit: This mapping is specific to Clean Core usage checks; it does not classify every ATC finding.

Related questions: CORE-01, CORE-02, CORE-03, CORE-04, CORE-05, CORE-06, CORE-09, CORE-13, CORE-16, CORE-18, GOV-01, GOV-05, GOV-06, GOV-14, GOV-15, GOV-16, GOV-18

SAP Clean Core best practices

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

RAP transaction and workflow boundary



Separate the local transaction from event delivery and the later workflow callback.

Explain it: What is lost if a handler calls a committing legacy API?

Limit: Exact RAP save methods and supported event infrastructure depend on scenario and release.

Related questions: RAP-04, RAP-05, RAP-15, RICEFW-11, RICEFW-12, RICEFW-14, SRAP-02, SRAP-03, SRAP-15, SRAP-18

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

SAP explicit and implicit enhancement options

Extension placement decision

Choose an extension approach

Start with the business requirement, supported contracts and transaction boundary

First	**Can standard functionality or configuration meet the need?** Use the standard capability and avoid unnecessary custom code.
Key user	**Does a delivered business context cover the adaptation?** Use supported custom fields, logic or UI adaptation tools.
On-stack	**Does the logic need local transactional consistency?** Use released local contracts and suitable ABAP Cloud extensibility.
Side-by-side	**Does the process need an independent or cross-system app?** Use supported remote APIs and events; design retries and consistency.

Placement does not determine the Clean Core level. Check dependencies separately.

Start from the business need and transaction boundary before choosing an extension location.

Explain it: When would local transactional consistency favor an on-stack extension?

Limit: Placement does not establish Clean Core level or availability in a specific edition.

Related questions: CORE-10, CORE-11, CORE-15, CORE-18, BTP-01, CLOUD-01, CLOUD-09, CLOUD-14, GOV-07, GOV-08

SAP ABAP Cloud development model

SAP Clean Core best practices

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

Internal table lookup

Find key 50: three internal-table access paths

The same seven records; choose the table and key for the actual access pattern.

STANDARD

Linear free-key example

Row	Key	Check
1	30	1
2	70	2
3	10	3
4	20	4
5	50	match
6	40	
7	60	

Inspect rows until the match.

SORTED

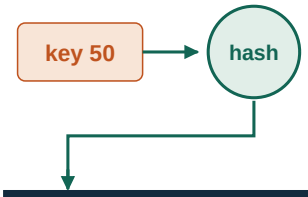
Ordered key lookup

Row	Key	Probe
1	10	
2	20	
3	30	
4	40	1st
5	50	3rd
6	60	2nd
7	70	

Illustrative probes: 40, 60, 50.

HASHED

Complete unique key



Bucket	Record key
...	...
h(50)	50 [match]
...	...

Bucket route is conceptual.
No primary index access.

Also consider secondary keys, inserts, memory, and range access.

Not a timing benchmark: sorted probes and hash buckets illustrate the lookup idea.

A free-key linear search, an ordered lookup, and a complete hashed-key lookup find the same record through different access paths.

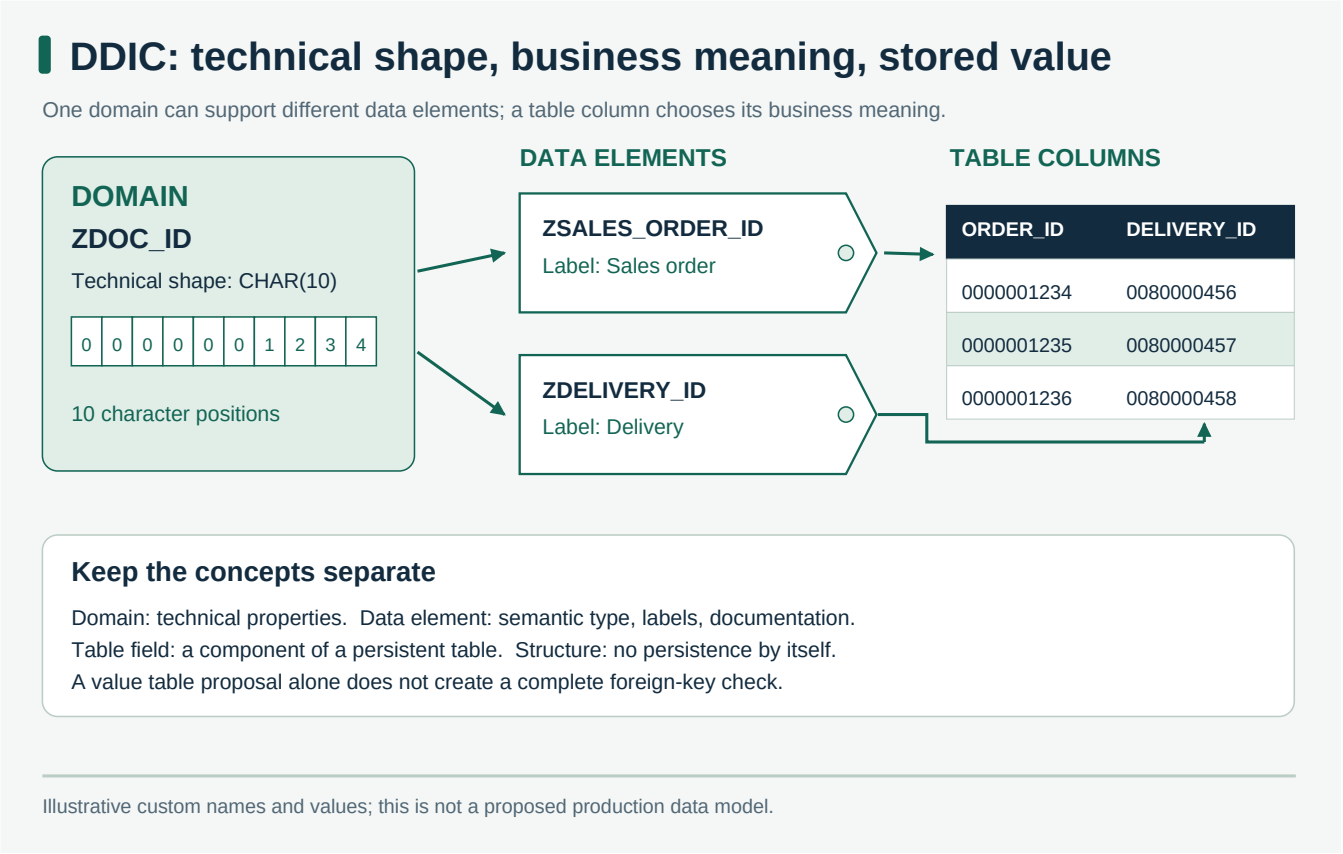
Explain it: What changes if I need all rows in a key range, or preserve input order?

Limit: A standard table can use secondary keys; a hash lookup requires a suitable complete key. Hash storage and probe order are schematic, not SAP implementation guarantees.

Related questions: ABAP-01, ABAP-02, ABAP-03

SAP internal table reference and examples

DDIC technical and semantic layers



A ten-character domain acts like a technical mold; data elements give two fields different business meanings, and table columns hold values.

Explain it: Which layer supplies field labels, and which object actually persists records?

Limit: The picture simplifies DDIC typing and does not imply that domain values or a value table enforce every database write.

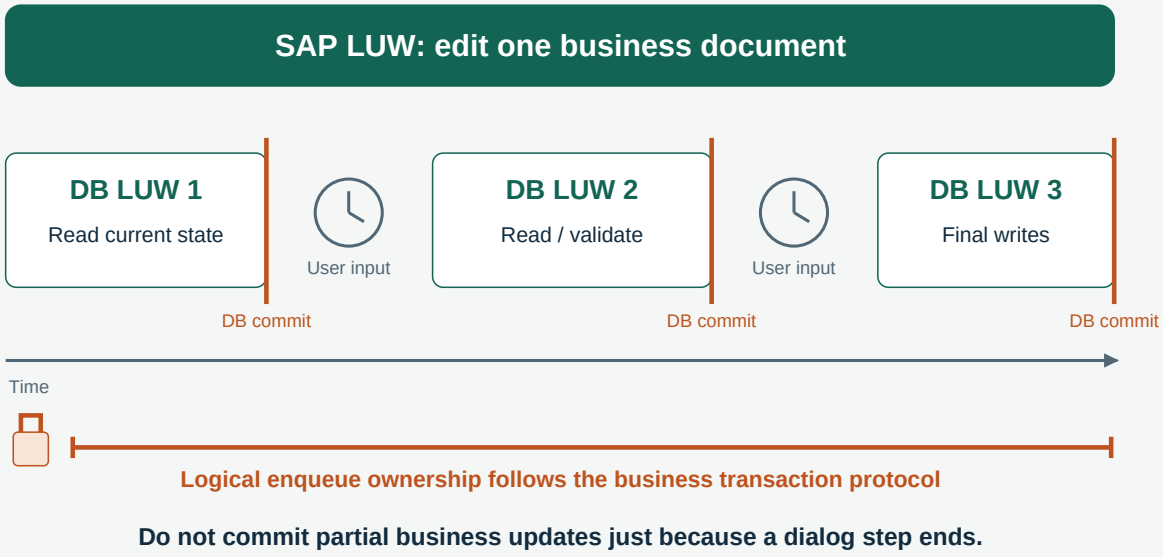
Related questions: ABAP-05, ABAP-06

SAP internal table reference and examples

SAP LUW and database LUW

One business change can span several database LUWs

Classic dialog example: intermediate reads, user interaction, then a coordinated final save.



Do not commit partial business updates just because a dialog step ends.

Simplified classic pattern. Update-task ownership and RAP's framework-controlled save need separate rules.

Several database transactions can serve a single classic SAP business transaction. The illustrated early database transactions only read; coordinated persistence happens at the end.

Explain it: Who owns the commit, and why does a database lock not protect a long user interaction?

Limit: The final save may use update processing in another context. Lock scope and transfer rules vary; the line is conceptual ownership, not a universal enqueue lifetime. Do not apply classic COMMIT WORK patterns inside RAP handlers.

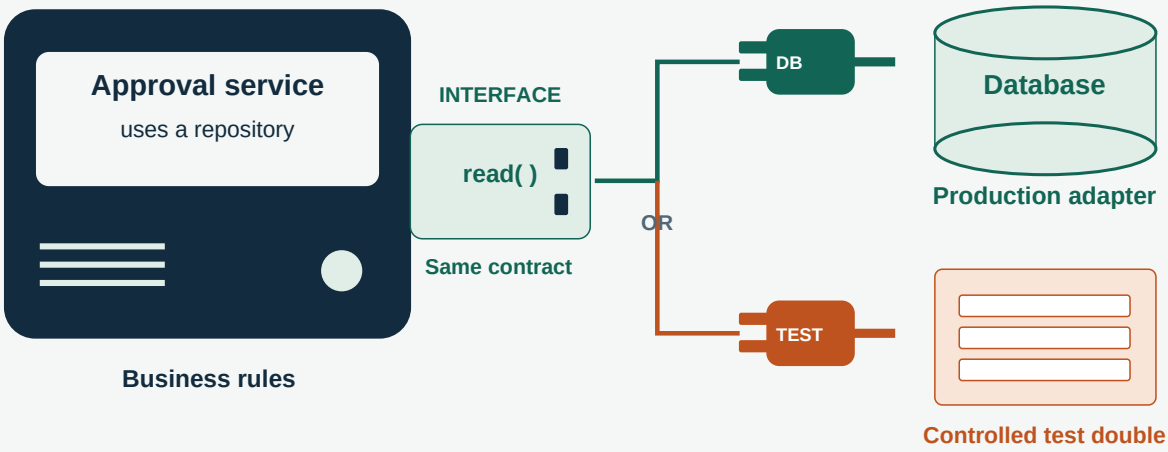
Related questions: ABAP-07, ABAP-08, ABAP-09

SAP internal table reference and examples

OO interfaces and dependency injection

One contract; production and test can plug in

Constructor injection supplies a collaborator. Polymorphism selects its implementation at runtime.



A test checks service behavior; a real integration check still verifies database behavior.

Inject one suitable implementation per instance. An adapter does not make an unreleased API released.

The service depends on a repository contract. Production and test implementations fit the same socket; the composition boundary supplies the appropriate collaborator.

Explain it: Which object chooses the implementation, and what does a double-based test fail to prove?

Limit: The plug is a design metaphor, not a SAP connector or code signature. Injection does not itself guarantee a good abstraction or production integration.

Related questions: OO-01, OO-02, OO-07, OO-10, OO-14, CLOUD-15

SAP ABAP Cloud development model

SAP Clean ABAP style guide

SQL join multiplication

A join repeats the header once for every matching item

Example: sum the order totals before and after a one-to-many join. All amounts are EUR.

ORDERS: 2 rows

Order	Header total	Currency
10	100	EUR
20	80	EUR

ITEMS: 3 rows

Order	Item	Product
10	1	A
10	2	B
20	1	C

JOIN ON order ID

RESULT: 3 rows

Order	Item	Header total	Currency
10	1	100	EUR
10	2	100	EUR
20	1	80	EUR

100 100 80

Joined sum = 280

Actual order total = 180

Fix the grain: aggregate at the intended business level before combining results.

SUM(DISTINCT amount) is not a general fix: different orders can have equal totals.

Outer join caveat: a WHERE predicate on the missing right side can remove unmatched left rows.

Joining two order headers to three items repeats order 10's total, inflating a naive sum from EUR 180 to EUR 280. For outer joins, matching conditions in ON can preserve unmatched left rows, while right-side conditions in WHERE can remove them.

Explain it: What is one row of this result supposed to represent? Where should a language filter go for optional text?

Limit: This is an inner-join example. WHERE and ON placement must follow intended semantics; neither DISTINCT nor SUM(DISTINCT amount) repairs every incorrect join or aggregation.

Related questions: SQL-04, SQL-06, SQL-07, MODEL-06, MODEL-10, MODEL-18

SAP RAP authorization control

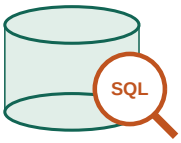
SAP ABAP SQL examples and semantics

Performance diagnosis

Diagnose the cost before choosing the optimization

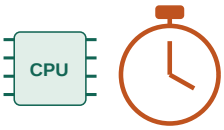
Collect evidence at the expensive boundary; the instruments below show no measured speed claims.

DATABASE



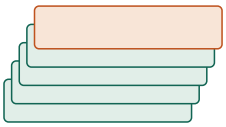
- Trace + execution plan
- Calls, rows, total cost
- ST05 / SQL Monitor
- Inspect filters / joins

ABAP CPU



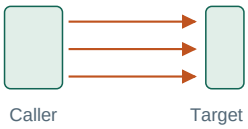
- Runtime + call tree
- SAT / supported profiler
- Loops, copies, calls
- Find repeated work

ROWS / MEMORY



- Inspect result volume
- Needed rows / columns?
- Bounded payload?
- Check memory growth

ROUND TRIPS



- Trace request count
- Latency and payload
- Repeated small calls?
- Check safe batching

Baseline > Locate the cost > Change one cause > Re-measure

Use representative data and verify identical business results after the change.

Tools depend on edition and permissions. HANA, a join, or a new index is not a universal speed guarantee.

Inspect database work, ABAP computation, transferred volume, and repeated calls as different possible causes. Choose the trace and change based on evidence.

Explain it: Is the largest cost one slow statement, many cheap statements, ABAP processing, or transfer volume?

Limit: No percentages or speedups are measured here. These cost categories can overlap; do not sum overlapping trace durations. Available tools differ between classic and cloud editions.

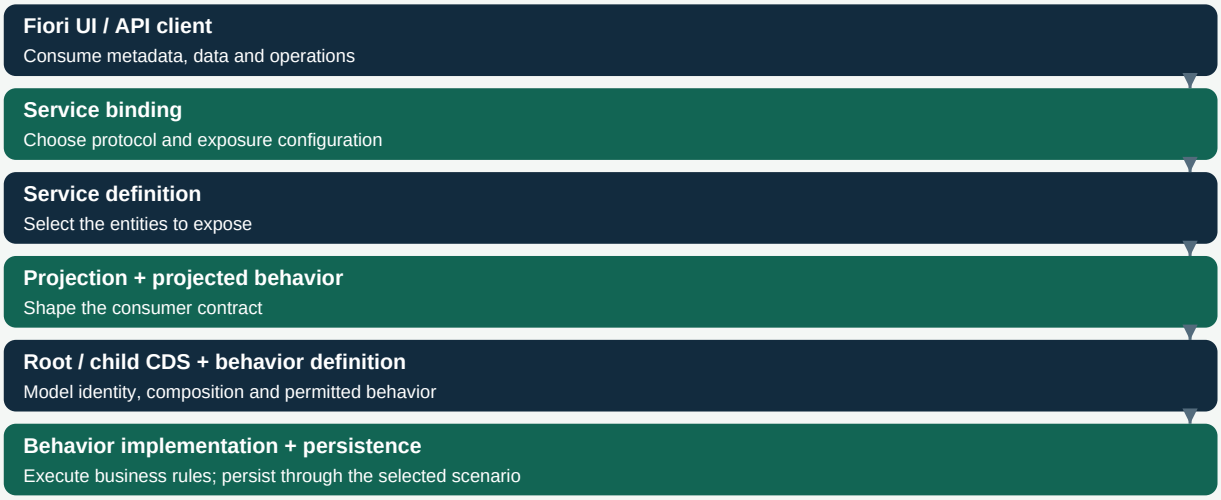
Related questions: SQL-09, SQL-10, SQL-11, SQL-12

SAP ABAP SQL examples and semantics

RAP contract stack

A service is a stack of contracts

RAP artifacts: each layer has a different job



A CDS entity alone is not a complete transactional service. A binding is not a behavior implementation.

Read from the client down to storage, then explain what each artifact contributes.

Explain it: Where would you restrict the exposed operation set?

Limit: Conceptual layers; the exact artifact set depends on the scenario and release.

Related questions: RAP-01, RAP-14, ADV-09, CDS-05, CDS-08, ODATA-02, CLOUD-10, MODEL-08, MODEL-16, SRAP-16, SRAP-22

SAP ABAP Cloud development model

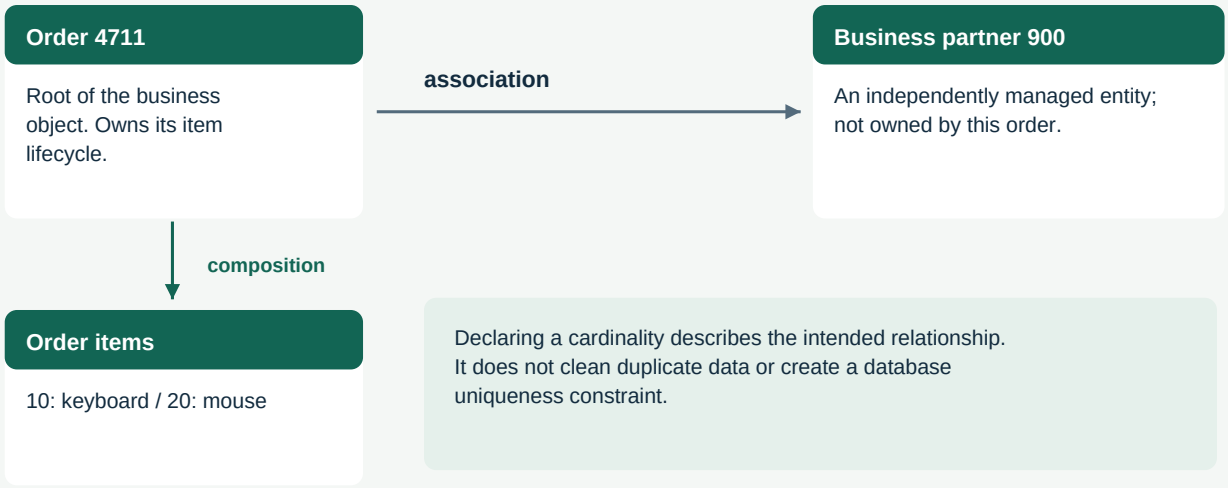
SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

CDS links and ownership

Association is a link; composition is ownership

An order example separates navigation from the business object boundary



Model [0..1], [1..1] or to-many relationships from actual data and the intended join conditions.

Items belong to the order BO; a partner reference crosses to an independently managed object.

Explain it: What breaks if a declared to-one relationship actually returns two partners?

Limit: Composition semantics and generated operations still require the corresponding RAP model.

Related questions: CDS-03, CDS-04, CDS-11, SQL-07, CLOUD-07, MODEL-06, MODEL-10

SAP ABAP Cloud development model

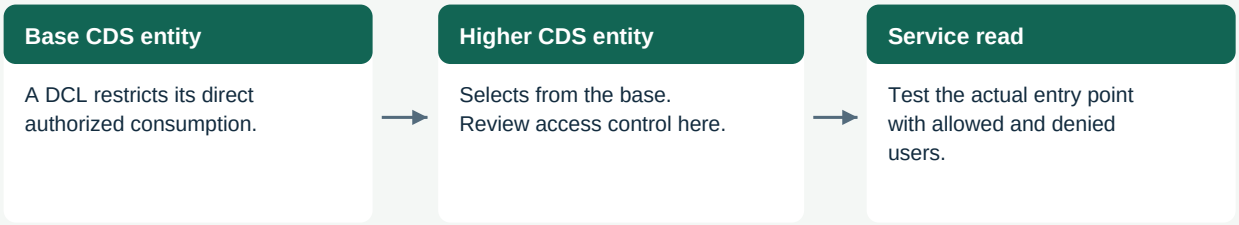
SAP Learning on CDS concepts

SAP RAP authorization control

CDS security boundaries

The exposed entity needs its own access decision

A base-layer DCL is not automatically inherited as an access filter



Mechanism	Purpose
DCL	Read access for the consumed CDS entity
RAP authorization	Permission to execute operations
Feature control	Availability in the current context

@AccessControl.authorizationCheck: #CHECK is not an authorization rule by itself.

Review security where consumers enter the model, rather than assuming that lower layers protect every upper view.

Explain it: Which two tests prove that an exposed service rejects unauthorized access?

Limit: Authorization behavior depends on the access path and supported DCL inheritance mechanisms.

Related questions: CDS-07, ADV-02, TEST-06, MODEL-01, MODEL-11, MODEL-14, MODEL-16

SAP Learning on CDS concepts

SAP RAP authorization control

SAP Learning on authorization

RAP buffer with a concrete value change

A buffer is a working set, not a second commit owner

The same business object can have different in-memory and saved states

Saved database	Value
Order 4711	100 EUR
Status	Open

modify →

Current RAP buffer	Value
Order 4711	120 EUR
Status	Open

Transactional read

Use EML for BO semantics and the current transactional view.

Completion owner

Framework or external EML consumer requests transaction completion.

Behavior handlers and savers must not issue COMMIT WORK or COMMIT ENTITIES themselves.

During modification, the transactional view can show 120 EUR while saved data still shows 100 EUR.

Explain it: Why can a direct SQL read be the wrong way to read the BO during modification?

Limit: Simplified single-record example, not a promise about a particular SQL isolation level.

Related questions: RAP-03, RAP-04, RAP-05, RAP-08, RAP-15, SRAP-01, SRAP-17, SRAP-19

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

RAP hook responsibilities

Four hooks answer four different questions

Choose by responsibility and supported trigger, not by an assumed method order

Precheck: accept this request?

Check incoming changes before they are applied to the transactional buffer. Example: reject an invalid requested transition.

Determination: derive what?

Derive values after relevant triggers. Example: calculate a default or dependent field. Design repeated execution to be safe.

Validation: may this be saved?

Check consistency at supported validation points. Example: a mandatory approval reason must be present.

Action: do which business operation?

An explicit operation, such as Submit. It may modify state and return a result under its declared contract.

Unmanaged RAP determinations and validations have draft-specific support. Implement active-instance checks in supported handlers/save logic.

Read each card as a separate design question; this is not an execution sequence.

Explain it: Where would you derive a total, reject an invalid input and implement Submit?

Limit: Check scenario and release support; do not rely on an undocumented ordering of independent hooks.

Related questions: RAP-06, RAP-13, ADV-04, ADV-05, SRAP-02, SRAP-03, SRAP-11, SRAP-12, SRAP-19

SAP Learning on behavior implementation options

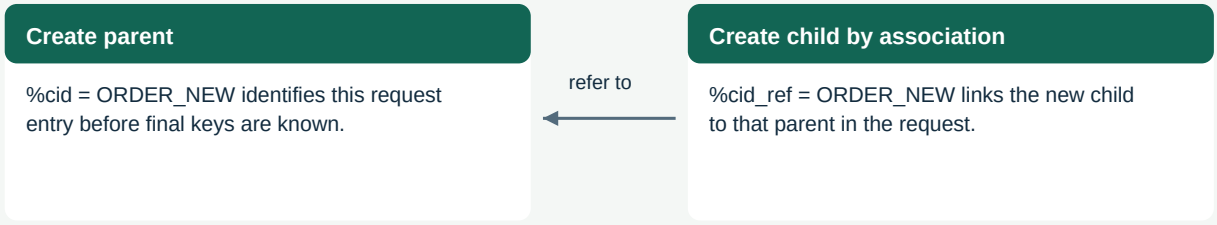
SAP: General RAP BO implementation contract

SAP Learning on authorization

EML identity tags

EML identities solve different problems

A parent and child created in the same request



Token	Meaning
%tky	Transactional identity; includes required context
%pid	Preliminary identity for late-numbering scenarios
%pre -> %key	Map preliminary identity to final keys in adjust_numbers
MAPPED	Return correlations and assigned identities

%cid is not a database key. %tky must not be reconstructed by blindly copying only the business key.

Content IDs connect entries within a request. Transactional and preliminary identities address other lifecycle needs.

Explain it: How can a new item refer to a newly created order before the final order number exists?

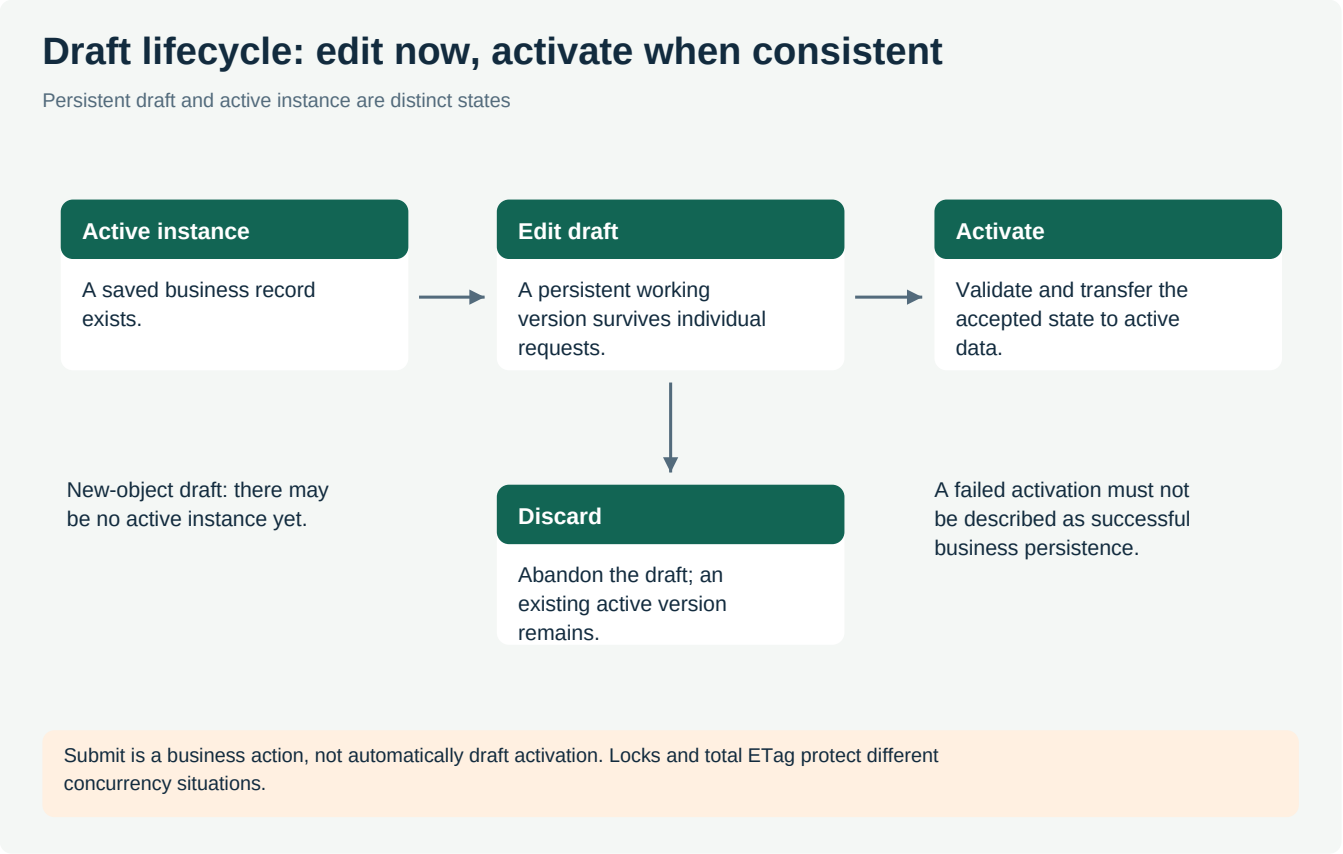
Limit: Generated EML types and available components are authoritative for the specific BO.

Related questions: RAP-07, RAP-10, RAP-11, SRAP-07, SRAP-08, SRAP-13

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

Draft lifecycle



Follow edit, activate and discard for an existing active object, then explain how a new-object draft differs.

Explain it: What remains after discarding an edit draft? What if activation fails?

Limit: Schematic state transitions; draft actions and supported flows depend on the BO and consumer.

Related questions: RAP-12, ADV-01, ADV-08, RICEFW-11, SRAP-09, SRAP-10, OPS-04

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

SAP Learning on authorization

Identity, authorization and feature control

Three gates: identity, permission and availability

A disabled button is not a security boundary

Authentication	Authorization	Feature control
Who is the caller? Establish a verified identity.	May this caller perform the operation? Check global and instance permissions.	Is the operation currently enabled by the business state?

1	2	3

Order state	Submit feature	Permission
Open	Enabled	Still required
Submitted	Disabled	Does not replace authorization

CSRF protects against request forgery in the relevant HTTP flow; it does not grant business authorization.

Treat identity, business permission and contextual availability as separate checks.

Explain it: If Submit is disabled in Fiori, what happens when someone calls the service directly?

Limit: Static operation characteristics differ from dynamic instance/global feature checks.

Related questions: ADV-02, ADV-03, ADV-13, ODATA-09, TEST-06, MODEL-02, MODEL-11, MODEL-12, MODEL-13, MODEL-16

SAP RAP authorization control

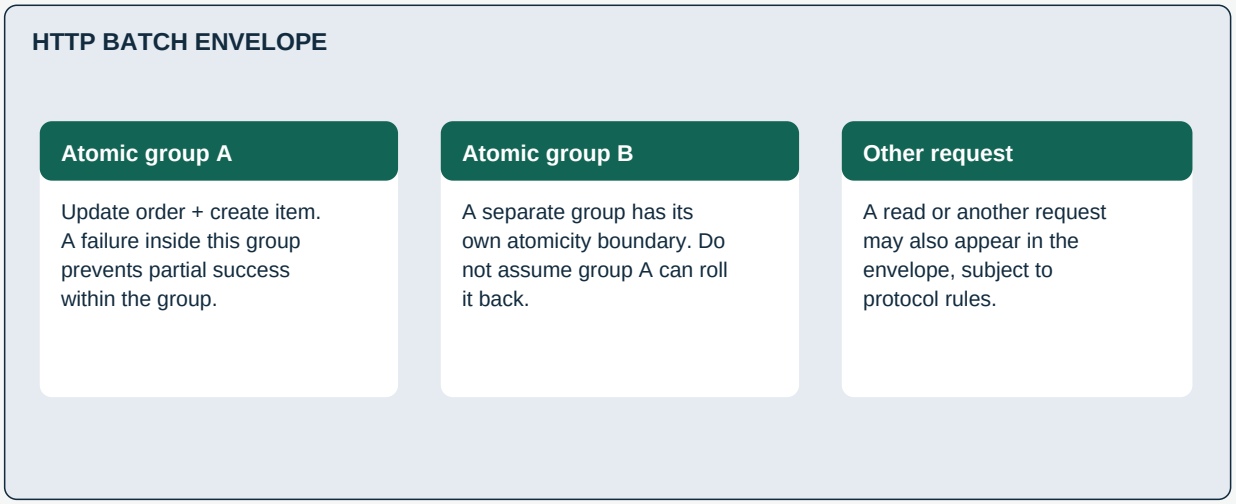
SAP Learning on authorization

SAP Learning on OData

OData batch envelope and atomic groups

One HTTP batch can contain several transaction groups

Think in atomic groups, not in envelopes



OData V2 changesets and OData V4 atomicity grouping are protocol-specific. Confirm service support and dependencies.

The outer envelope reduces HTTP packaging overhead; it does not establish one universal database transaction.

Explain it: If a later independent group fails, must an earlier successful group be rolled back?

Limit: Conceptual grouping, not an exact multipart or JSON batch payload.

Related questions: ODATA-07, BTP-07, TEST-07, SRAP-20

SAP: General RAP BO implementation contract

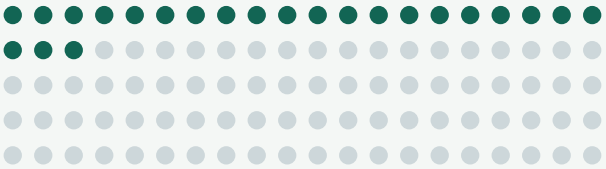
SAP Learning on OData

SAP ABAP Cloud FAQ

Filtering, paging and total count

Count describes the filtered set; a page is a slice

Example: 100 orders, 23 match, the current page contains 5



100 total orders
23 match the filter

Filter	Order	Page	Return
23 matching records	Stable and deterministic	Skip 10, take 5	5 rows + count 23

Respect requested fields, filters, sorting, paging and count in a custom query provider. Do not count only the returned page.

Count and paging answer different questions. Stable ordering matters when the client requests successive pages.

Explain it: For this example, what count belongs beside a page of five records?

Limit: Provider-side execution order may be optimized while preserving request semantics.

Related questions: ODATA-03, ODATA-04, ADV-10, ODATA-11, MODEL-03, MODEL-04, MODEL-05, OPS-07

SAP RAP authorization control

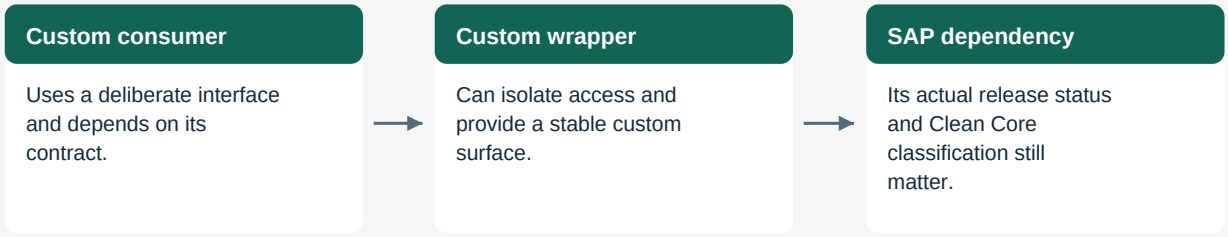
SAP Learning on authorization

SAP RAP test frameworks and boundaries

Clean Core wrapper dependency

A wrapper changes the contract, not its ancestry

Inspect the dependency behind the clean facade



Claim	Reality
Released custom surface	Does not retroactively release the SAP object
Runs side-by-side	Does not prove every consumed API is released
One Level A dependency	Does not classify all other dependencies

A-D grades are not a one-to-one renaming of the historical three-tier model. Check permitted use in the target edition.

Trace dependencies across the wrapper instead of classifying the solution from its visible surface alone.

Explain it: What evidence would let you accept a classic SAP dependency responsibly?

Limit: A governance review must consider technology restrictions, classifications and lifecycle risk.

Related questions: CORE-04, CORE-05, CORE-07, CORE-08, CORE-11, CORE-14, CORE-15, OO-10, CLOUD-03, GOV-03, GOV-07, GOV-14, GOV-15

SAP ABAP Cloud development model

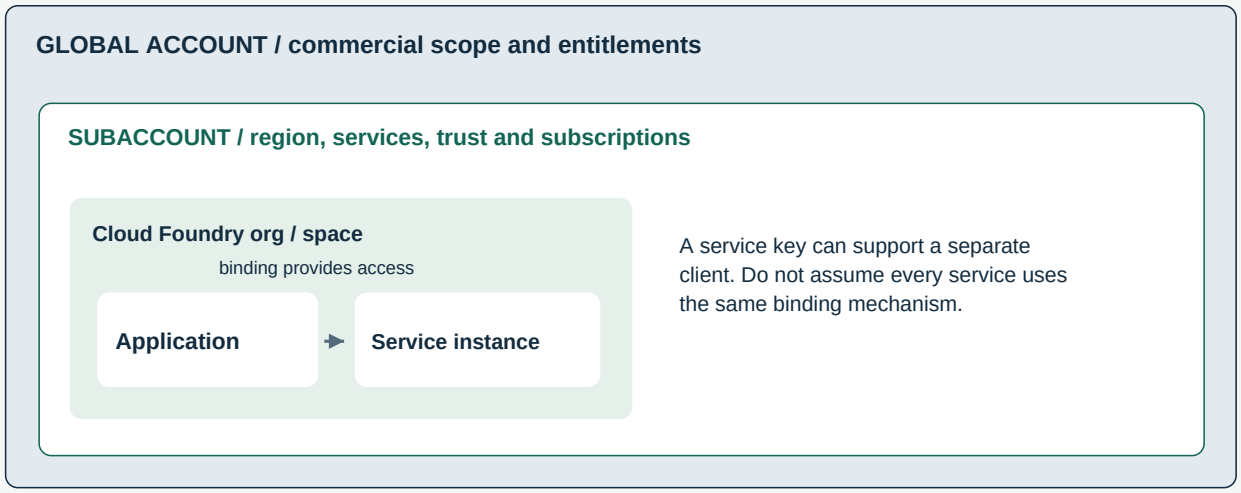
SAP Clean Core best practices

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

BTP account and runtime map

BTP organization and runtime are different axes

A schematic Cloud Foundry example



Cloud Foundry spaces do not describe every BTP runtime. Avoid placing service credentials in source control.

Read the nesting as an organizational example, then distinguish the application, service instance and credentials.

Explain it: How is a service instance different from a binding or service key?

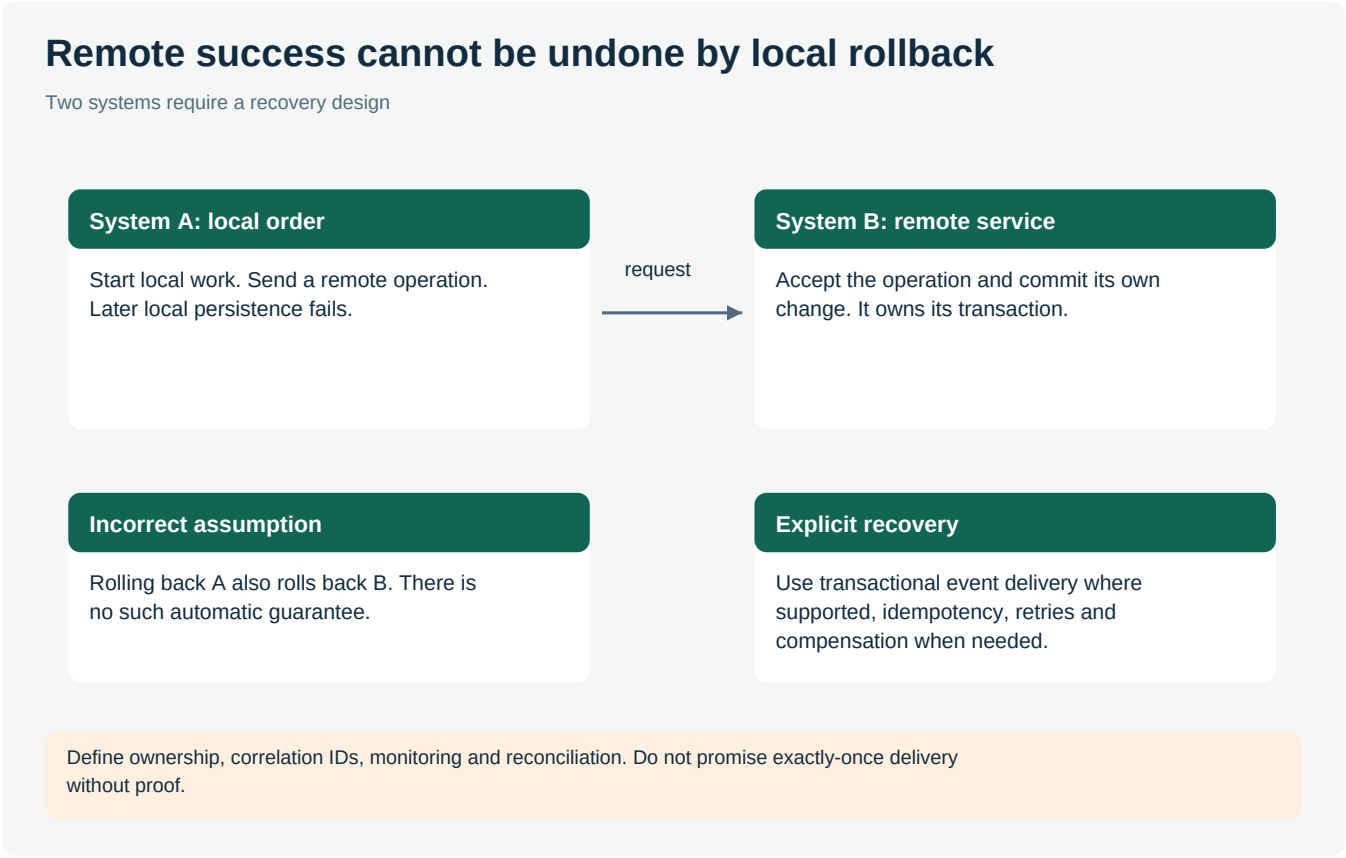
Limit: Schematic Cloud Foundry route; other BTP environments have different runtime structures.

Related questions: BTP-01, BTP-02, BTP-04, BTP-05, BTP-09, BTP-10, CLOUD-02, CLOUD-08

SAP ABAP Cloud development model

SAP ABAP Cloud FAQ

Local and remote transaction boundaries



The remote commit belongs to another system, so local failure needs an explicit recovery strategy.

Explain it: What should happen if the remote side succeeds but the response is lost?

Limit: This explains distributed consistency, not a particular messaging product guarantee.

Related questions: BTP-06, BTP-07, BTP-08, RICEFW-05, RAP-15, CLOUD-01, CLOUD-05, SRAP-04, SRAP-15, OPS-11

SAP ABAP Cloud development model

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

Duplicate events and stale approvals

One business effect, even if the message arrives twice

Idempotency needs a key, a state check and safe persistence

Delivery	Business event ID
Attempt 1	EVT-204
Retry	EVT-204

→ **Consumer transaction**

Check event identity and current state. Apply an allowed transition and record processing safely together.

Current state	Incoming request	Result
Pending / new event	Approve	Apply once
Approved / same event	Approve again	Return prior outcome or no-op
Changed / stale version	Old approval	Reject or reconcile; do not overwrite

A check-then-write race is not solved by an in-memory set. Enforce the concurrency and persistence boundary.

The event identity and current business state decide whether to apply, ignore or reject the request.

Explain it: Two consumers process the same event simultaneously: what prevents a double effect?

Limit: Exact transactional deduplication mechanisms depend on the chosen infrastructure.

Related questions: BTP-08, RICEFW-11, RICEFW-12, RICEFW-13, ADV-11, OO-12, SRAP-18, SRAP-23, OPS-11

SAP: General RAP BO implementation contract

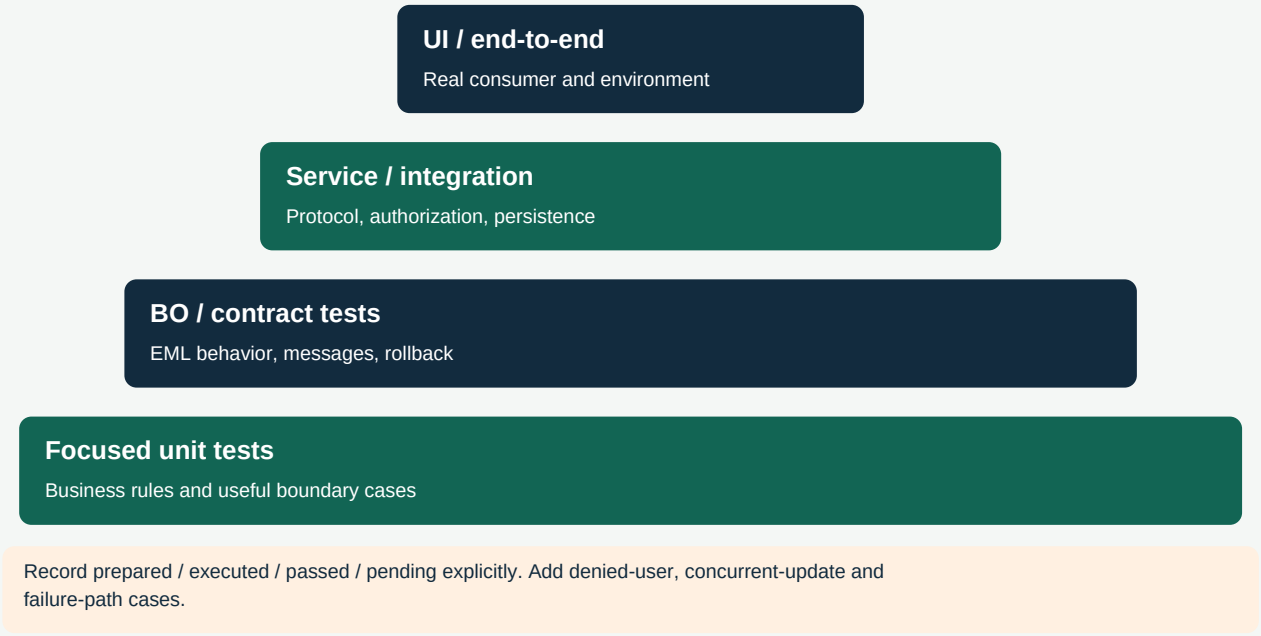
SAP Learning on authorization

SAP RAP test frameworks and boundaries

Testing scope and evidence

Different tests produce different evidence

A passing test double does not prove a deployed service works



Record prepared / executed / passed / pending explicitly. Add denied-user, concurrent-update and failure-path cases.

The widening base suggests many focused checks, but there is no mandatory numerical ratio.

Explain it: Which risk remains after an ABAP Unit test with doubles passes?

Limit: Conceptual coverage map, not a count of tests actually run in SAP.

Related questions: TEST-01, TEST-02, TEST-03, TEST-04, TEST-05, TEST-07, TEST-08, TEST-09, TEST-10, OO-14, CLOUD-18, OPS-01, OPS-02, OPS-09, OPS-14, OPS-15, OPS-16, OPS-18

SAP ABAP Cloud development model

SAP RAP test frameworks and boundaries

SAP Clean ABAP style guide

Explain decisions with evidence

A strong interview answer has a visible chain of evidence

Build a truthful story that an interviewer can follow

1	**Context**	What business problem and constraints mattered?
2	**Decision**	What did you choose, and which alternative did you reject?
3	**Mechanism**	How did the code or architecture produce the behavior?
4	**Evidence**	What did you measure or test? State actual results.
5	**Limit**	What remains uncertain, release-specific or incomplete?

Use your own experience. If you only prepared a test or proposed a design, say so explicitly.

Practice a 60-second version first, then expand the mechanism and evidence when prompted.

Explain it: Can you name the decision, trade-off and evidence without inventing metrics?

Limit: A rehearsal structure, not a scripted personal history or a substitute for technical depth.

Related questions: STORY-01, STORY-02, STORY-03, STORY-04, STORY-05, STORY-06, TEST-10, OPS-18

SAP RAP test frameworks and boundaries

SAP Clean ABAP testing guidance

HTTP request and response anatomy

Read an HTTP exchange as a contract

Illustrative HTTP/1.1 layout; HTTPS protects the transport

REQUEST: client to service

GET /orders/4711 HTTP/1.1

Host: api.example.test

Accept: application/json

Method + target identify the request.

Accept asks for a representation.

Credentials are intentionally omitted.

RESPONSE: service to client

HTTP/1.1 200 OK

Content-Type: application/json

ETag: "v7"

{"id":"4711","status":"Open"}

ETag is an opaque version validator.

Transport / HTTP → **Representation / JSON**

REST is an architectural style; JSON and XML are payload formats. OData adds service conventions on top of HTTP.

Read the method, target, status and headers before interpreting the body. The sample domain is deliberately non-production.

Explain it: Which header describes the returned body, and which requests a response format?

Limit: A teaching example, not a runnable endpoint. HTTP/2 and HTTP/3 use different wire framing.

Related questions: HTTP-01, HTTP-02, HTTP-05, HTTP-06, HTTP-10, ODATA-02

SAP Learning on OData

HTTP semantics: IETF RFC 9110

JSON and XML: values and mapping

Same business values, two representation contracts

Do not confuse an identifier with a number or an absent value with null

JSON example

```
{
  "id": "0004711",
  "currency": "EUR",
  "amount": "120.50"
}
```

XML example

```
<order id="0004711">
  <currency>EUR</currency>
  <amount>120.50</amount>
</order>
```

Case	Required decision
"0004711"	Keep the leading zeros: this is an identifier
Missing / null / empty	Specify their meanings, especially for partial updates

The amount is a string in this example by contract. Neither JSON nor XML automatically defines business precision or validation.

These are two explicitly designed representations of the same sample values, not a universal automatic JSON/XML conversion.

Explain it: What could go wrong if an ABAP mapper converts the identifier to an integer?

Limit: XML namespace/schema rules and JSON API types must be defined by the actual interface contract.

Related questions: HTTP-03, HTTP-07, HTTP-08, HTTP-09, HTTP-12, ABAP-12

SAP internal table reference and examples

HTTP semantics: IETF RFC 9110

RFC modes and guarantee boundaries

RFC modes change execution and delivery semantics

Choose the required boundary before choosing the syntax

Mode	Key idea	Interview distinction
sRFC	Caller waits for the result	Remote call is not local atomicity
aRFC	Caller can continue	Collect results; no durable retry by itself
tRFC	Transactional delivery unit	Runtime/TID scope; not global ordering
qRFC	Queue orders units	Ordering is scoped to configured queues
bgRFC	Background units / queues	Check type, configuration and monitoring

A BAPI is a business interface contract. RFC is a call mechanism. A background job is a scheduling concept.

Transport-level guarantees do not prove exactly-once business effects across arbitrary external systems or manually repeated submissions.

Compare the caller behavior and delivery unit, then name the monitoring and retry owner.

Explain it: Why does an asynchronous call not automatically become a durable queued transaction?

Limit: Classic terminology; API release, runtime availability and transaction rules must be checked for the target.

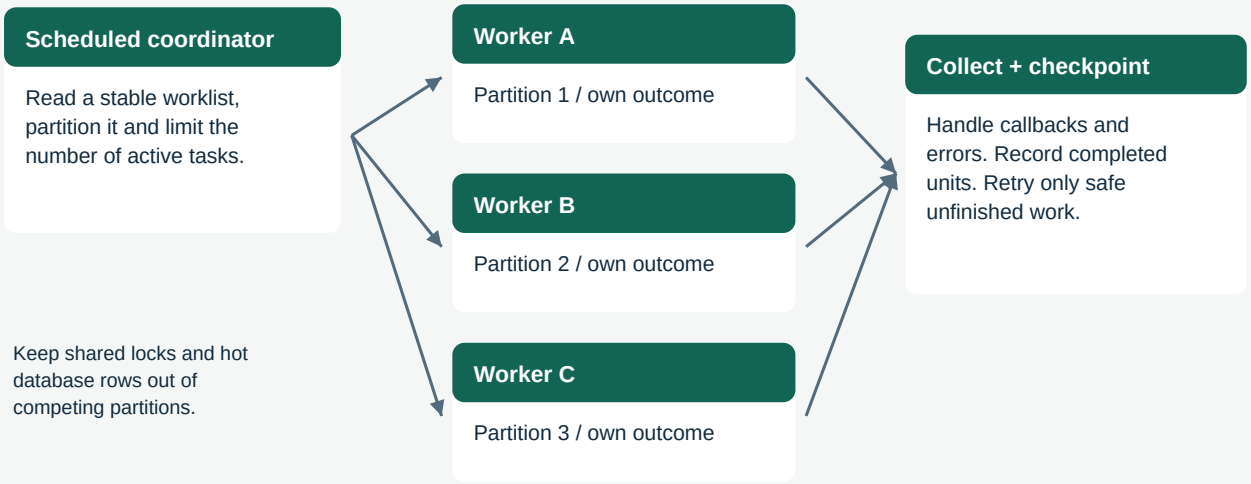
Related questions: RFC-01, RFC-02, RFC-03, RFC-04, RFC-05, RFC-08, RFC-12

SAP RFC variants and their quality-of-service scope

Parallel processing inside a job

A job can coordinate parallel workers

Scheduled execution and concurrent work are independent design choices



Bound concurrency by available resources. A finished job status does not prove every business item succeeded. WAIT can affect the DB LUW.

A coordinator schedules bounded independent work and collects explicit outcomes; a scheduler does not create parallelism automatically.

Explain it: If worker B fails after A commits, what is safe to retry?

Limit: Conceptual fan-out/fan-in. Worker session and transaction boundaries require explicit design.

Related questions: RFC-06, RFC-07, RFC-08, RFC-09, RFC-10, RFC-11, RFC-12, OPS-17

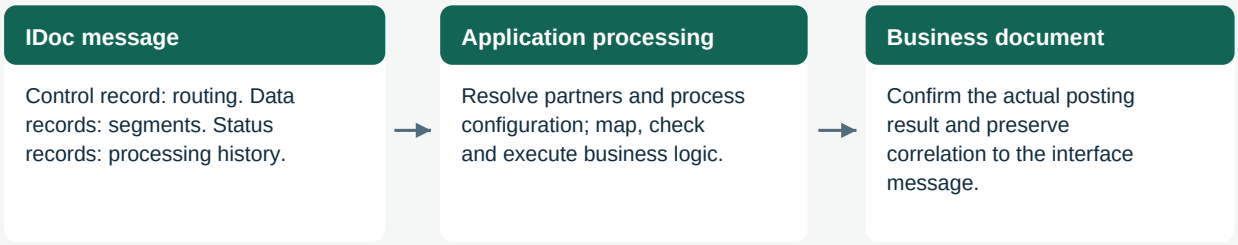
SAP RAP test frameworks and boundaries

SAP RFC variants and their quality-of-service scope

IDoc processing and AIF operations

IDoc transports a business message; AIF helps operate it

AIF support depends on the configured interface and deployment



AIF: configured monitoring and error-handling layer

Show business keys and errors to authorized recipients. Diagnose and correct the cause, then reprocess through the owning mechanism. Prevent a second business effect.

OData is a service protocol, IDoc is a message/interface technology, and AIF is a framework. They are not three interchangeable transports.

Follow the message to the business outcome. The AIF band represents supported configured capabilities, not an obligatory stage for every IDoc.

Explain it: Before restarting an error, how would you check whether a document already exists?

Limit: Do not infer end-to-end business success from a transport hand-off status or assume all interfaces use AIF.

Related questions: INTF-01, INTF-02, INTF-03, INTF-04, INTF-05, INTF-06, INTF-07, INTF-08, INTF-09, INTF-10, INTF-11, INTF-12, RICEFW-06

SAP explicit and implicit enhancement options

SAP IDoc control record, message type, basic type, and extension

EML update intent: absent versus clear

An initial value is not the same as an omitted update

Worked EML mapping: saved Comment = "Urgent"; saved Amount = 120 EUR

Update intent	Comment value	%control-Comment	Resulting Comment
Leave unchanged	Initial / blank	off	Urgent
Explicitly clear	Initial / blank	on	(empty)
Replace text	Follow up	on	Follow up

Same value, different intent

Blank + off preserves the old text.
Blank + on requests a clear.

Untouched fields stay untouched

Amount remains 120 EUR in all three cases.
Do not turn every control flag on.

Use the generated field control or UPDATE FIELDS contract. Trace payload mapping and determinations; do not use IS INITIAL as a universal presence test.

The first two requests carry the same initial ABAP value but differ in update intent. The field control decides whether the old comment is preserved or explicitly cleared.

Explain it: How would you preserve Amount while clearing Comment, and how could an adapter lose that distinction?

Limit: This is a worked mapping example. Actual generated EML types, UPDATE FIELDS syntax, protocol nullability, and business validation remain authoritative.

Related questions: SRAP-06, SRAP-12, HTTP-07, HTTP-09

SAP: General RAP BO implementation contract

HTTP semantics: IETF RFC 9110

FAILED, REPORTED, and MAPPED in a worked create

Three response channels; three different meanings

Illustrative create interaction: C1 and C3 accepted into the buffer; C2 rejected

Input %cid	Amount	Interaction outcome
C1	100 EUR	Accepted; assigned key 4711
C2	-5 EUR	Rejected: amount must be positive
C3	80 EUR	Accepted; key 4712; informational message

FAILED	REPORTED	MAPPED
C2: rejected create	C2: amount error	C1 -> 4711
	C3: information	C3 -> 4712
Identity + failure cause	Message + instance context	Creation / key correlation

Interaction responses are not a commit receipt. Inspect the subsequent save outcome.

A REPORTED message is not always a failure; MAPPED is not a universal success list. The consumer and request boundary determine the atomicity policy.

Three input identities produce separate rejection, message, and key-correlation information. A message on C3 does not itself reject that input, and assigned keys do not prove durable persistence.

Explain it: What must the consumer inspect after this interaction, and what happens if the later save fails?

Limit: Conceptual response contents, not executable EML. Available fields and correlation depend on the operation and BO; save and atomicity semantics must be tested separately.

Related questions: SRAP-05, SRAP-07, SRAP-20, OPS-03

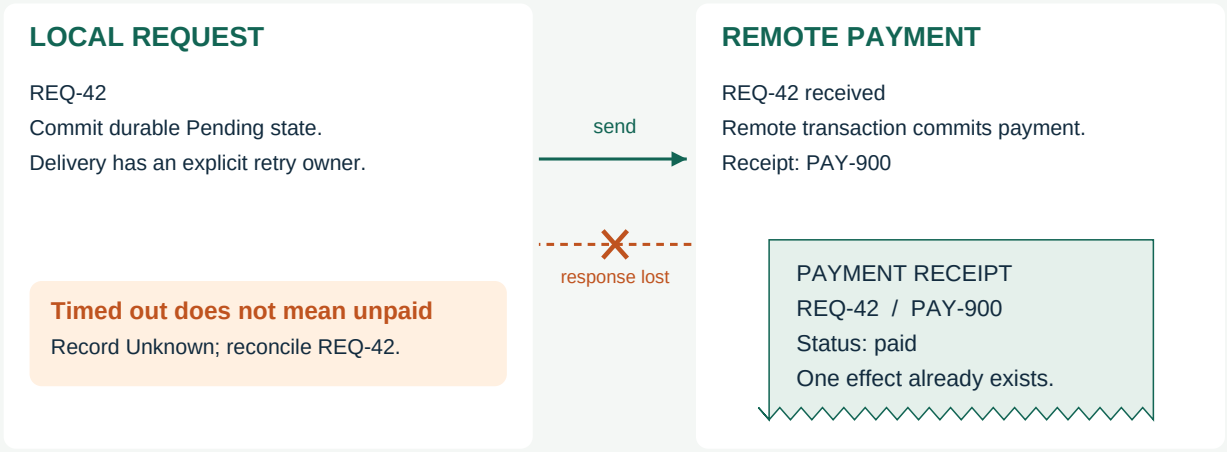
SAP: General RAP BO implementation contract

SAP RAP test frameworks and boundaries

Remote commit and uncertain local outcome

A lost response leaves an unknown outcome

Worked recovery: stable request key REQ-42; remote payment receipt PAY-900



Recover by correlation: query status or use the supported idempotency contract.

A local rollback cannot erase PAY-900. Blindly resending a non-idempotent operation can create another payment; compensation is a separate business operation.

The local application durably records intent before separately owned delivery. The remote payment commits, but its reply is lost; recovery must identify the existing effect through REQ-42.

Explain it: How do you distinguish an unprocessed request from a committed payment whose response never arrived?

Limit: The delivery and idempotency mechanisms are design requirements, not automatic guarantees from HTTP or RAP. This example does not claim distributed atomicity.

Related questions: SRAP-04, SRAP-18, SRAP-23, OPS-11, CLOUD-01

SAP ABAP Cloud development model

SAP: General RAP BO implementation contract

SAP RAP test frameworks and boundaries

Existing and proposed state authorization

Permission for the old state does not approve the new state

Policy fixture: user may edit organization A, but may not transfer an order to B

Order	Existing organization	Proposed organization	Decision
4711	A	A	Allowed if other rules pass
4712	A	B	Reject target change
4713	A	(omitted)	Keep A; check actual intent



Use the supported authorization and incoming-change checks for the scenario. A hidden or disabled UI field is not the backend security boundary.

Editing the current A instance and moving it into B are different authorization decisions. Partial input must be interpreted with existing state without treating omitted fields as explicit clears.

Explain it: Which state does each check inspect, and what happens if a determination changes the target scope?

Limit: A policy example, not a universal company-code rule. Implement through the target RAP scenario's supported hooks; prechecks do not replace the complete authorization model.

Related questions: MODEL-02, MODEL-13, MODEL-16, SRAP-12

SAP: General RAP BO implementation contract

SAP RAP authorization control

Secure custom-query filtering, count, and page

One authorized predicate must feed both data and count

Request: user may see A; amount > 100; order by ID; skip 1; take 2

SOURCE: 6 rows

ID	Org	Amount
1	A	80
2	A	120
3	B	200
4	A	150
5	B	300
6	A	180

Do not flatten an unsupported Boolean filter.

Translate it correctly or reject it.

LOGICAL RESULT CONTRACT

Authorization + request filter
Keep IDs 2, 4, 6

▼

Count the same filtered set
count = 3

▼

Deterministic order, then page
skip 1, take 2 -> IDs 4, 6

RETURN 2 rows: ID 4 and ID 6
RETURN total count 3, not 2 and not 6

Unauthorized rows must not leak through count or aggregation. Stable ordering needs a tie-breaker; concurrent changes require an explicit paging consistency contract.

The same authorized logical set contains IDs 2, 4, and 6. Counting it returns three; skipping one and taking two returns IDs 4 and 6 without revealing hidden B rows.

Explain it: What should a count-only request return, and what should happen if the filter cannot be represented safely?

Limit: The drawing specifies semantics, not a mandatory physical execution plan. A remote/custom provider must preserve equivalent authorization, filtering, count, and ordering behavior.

Related questions: MODEL-03, MODEL-04, MODEL-05, OPS-07

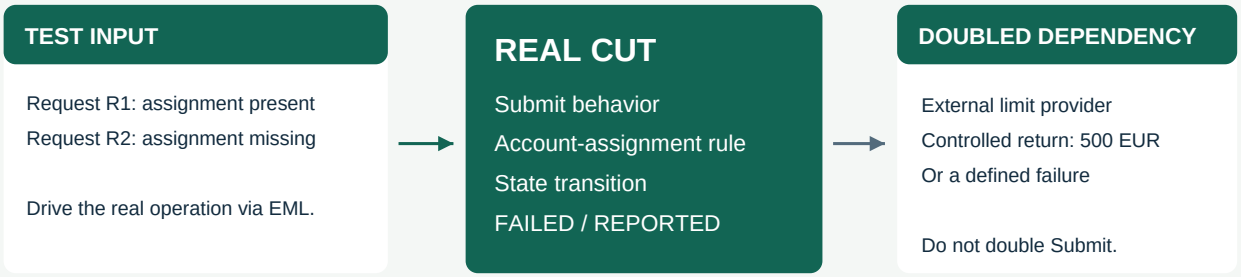
SAP RAP authorization control

SAP RAP test frameworks and boundaries

Real behavior under test and dependency doubles

Keep Submit real; double only its true dependencies

CUT = code under test. A configured mock response cannot prove the behavior it replaces.



Case	Required assertions
R1: valid	Expected state transition and correlated response
R2: invalid	Rejected identity, useful message, no forbidden state
Rule deliberately broken	The negative test must fail

Persistence using doubles is isolated evidence. Real database, service authorization, locking, and external integration still need targeted tests on the actual runtime.

Two deliberately different inputs exercise the real Submit rule while a separate limit-provider dependency is controlled. Assertions inspect state and response identity, not only execution coverage.

Explain it: If the account-assignment rejection is removed, which test fails? What evidence remains missing after these tests pass?

Limit: The fixture is conceptual and not an executed ABAP Unit result. Framework double support and persistence setup depend on the target release and test boundary.

Related questions: OPS-01, OPS-02, OPS-03, OPS-15, MODEL-14

SAP RAP authorization control

SAP RAP test frameworks and boundaries

Release contracts versus Clean Core levels

Release contract and Clean Core level answer different questions

C0 / C1 / C2 describe permitted purpose. A / B / C / D classify extension exposure.

WHAT USE IS RELEASED?

Contract	Purpose
C0	Extend
C1	Use system-internally
C2	Use as remote API

WHICH DEPENDENCIES / TECHNIQUES?

Level	Assessment focus
A	Released APIs / supported approach
B	Classified classic APIs
C	Internal objects
D	Non-recommended use

Worked check: a C1-released API is not a Level C API.

Confirm purpose, consumer visibility, language version, and the full dependency path.

A C1 local contract does not automatically release the same object for remote use.

ATC priorities belong to specific checks and configuration. Do not map every priority 1/2/3 finding universally to Clean Core D/C/B.

The two tables are separate classification axes. C1 identifies a local-use contract, while Level C identifies internal-object exposure in the Clean Core assessment context.

Explain it: What else must you verify after finding a C1 release flag, and why does a wrapper not erase the underlying exposure?

Limit: These are the three contracts requested for this comparison, not the complete release-contract catalogue. Exact object classification, extension technology, target edition, and ATC scope remain decisive.

Related questions: GOV-01, GOV-02, GOV-03, GOV-05, GOV-18, CLOUD-05

SAP ABAP Cloud development model

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

Exception governance and review lifecycle

An exception is a bounded decision, not a new API contract

Illustrative governance record: EX-017 permits one reviewed internal read dependency

EX-017 / reviewable exception record

Scope: adapter ZCL_LEGACY_READ, one read path

Reason: required field absent from released query

Owner: application team + named risk approver

Evidence: denied-user and result-total tests

Review: next platform upgrade or API successor

Exit: replace dependency; remove exemption

Status: approved scope, not unrestricted access

1 **ASSESS**
Verify gap, scope, alternatives.

2 **DECIDE**
Approve risk and required controls.

3 **RECHECK**
Trigger: code, API, or release change.

4 **CLOSE OR RENEW**
Show replacement or fresh evidence.

A new write call is outside EX-017. It needs a new decision, not silent inheritance.

An exemption can change finding handling; it cannot legalize unsupported ABAP Cloud syntax, release a SAP object, or prove transaction correctness.

A concrete exception record limits one dependency, states its evidence and owner, and defines review and exit triggers. A changed implementation must be assessed against that scope.

Explain it: Which change invalidates this decision, and what proves that an exception is resolved rather than merely hidden?

Limit: EX-017 and the class name are fictional teaching identifiers. Actual approvals, gate behavior, and allowed development techniques follow the target environment and organization policy.

Related questions: GOV-04, GOV-07, GOV-11, GOV-12, GOV-15, GOV-18

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

RAP implementation ownership matrix

Choose who owns interaction and persistence

Compare implementation responsibilities, not the age of the application

Scenario	Interaction buffer	Persistence
Managed	Framework	Framework
Managed + unmanaged save	Framework	Application saver
Unmanaged	Application	Application saver

Existing API fits?

Check transaction ownership, supported save phase, numbering and failure behavior.

Commit owner stays fixed

A behavior implementation cannot take over transaction completion with COMMIT WORK.

Additional save supplements managed persistence. It is not the same as replacing persistence with unmanaged save.

Trace the same request through three implementation choices and name the owner of each responsibility.

Explain it: Which part stays managed when only the save implementation is replaced?

Limit: Conceptual comparison; specific saver variants and failure support depend on release and scenario.

Related questions: RAP-02, SRAP-01, SRAP-02

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

RAP identity and numbering timeline

Early and late numbering: know which identity you hold

Example: a new order has request identity before its final document number

Moment	Early numbering example	Late numbering example
Create request	%cid = NEW_1	%cid = NEW_1
Interaction	Final key already assigned	Preliminary identity as configured
ADJUST_NUMBERS	No late assignment needed	Map preliminary to final key
After successful save	Consumer has saved key	Consumer resolves final key

A generated UUID and a gap-sensitive business document number solve different requirements. Do not return a preliminary identifier as a saved final document number.

Final key allocation is not proof of commit. Rollback can still affect the outcome; numbering and persistence contracts must be checked.

Compare when the identity becomes final and when persistence is confirmed.

Explain it: What must an external EML consumer inspect after MODIFY and after COMMIT?

Limit: Illustrative numbering paths; exact %pid/%pre mapping and conversion syntax are release/scenario-specific.

Related questions: RAP-10, RAP-11, SRAP-07, SRAP-08

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

Augmentation versus additional save

Augmentation adapts the request; additional save adds work

Two hooks at different boundaries, not interchangeable extension techniques

Projection request

Convenience input arrives. Augmentation enriches supported underlying modifications.

→

Managed persistence

The framework saves owned data. Additional save performs supported supplementary work.

Question	Augmentation	Additional save
Where?	Projection interaction	Save sequence
Example	Derive underlying request input	Write supported audit data

Neither hook authorizes an independent COMMIT. Choose by contract, not by an assumption that any hook can change any data.

Locate the input transformation and persistence supplement on the same conceptual timeline.

Explain it: Why would writing an audit row not make augmentation the right save hook?

Limit: The arrow is conceptual; availability, supported fields and trigger details depend on the target RAP release.

Related questions: ADV-06, SRAP-02, SRAP-14

SAP: General RAP BO implementation contract

SAP Learning on authorization

Two ETag boundaries

Entity ETag and total ETag protect different checks

One order root, one child change, two concurrency situations

Entity conditional update

Alice and Ben read the same entity version. Alice saves. Ben submits the old If-Match value.

Draft resume after lock expiry

A draft was based on active state. Another process changes an active child. Resume must detect a stale base.

Check	Protects	Expected response
Entity ETag	Same entity update version	Reject stale conditional update
Total ETag	Relevant active BO state	Detect stale draft on resume

Use two independent contexts for a real concurrency test. A child-only change must participate in the required total-ETag contract.

The left side tests a conditional entity update; the right tests the active-state basis of a resumed draft.

Explain it: Why does testing one entity ETag not prove the draft total-ETag implementation?

Limit: ETags are opaque. Verify lock expiry, relevant child changes and generated behavior on the actual system.

Related questions: ADV-01, SRAP-10, OPS-05

SAP: General RAP BO implementation contract

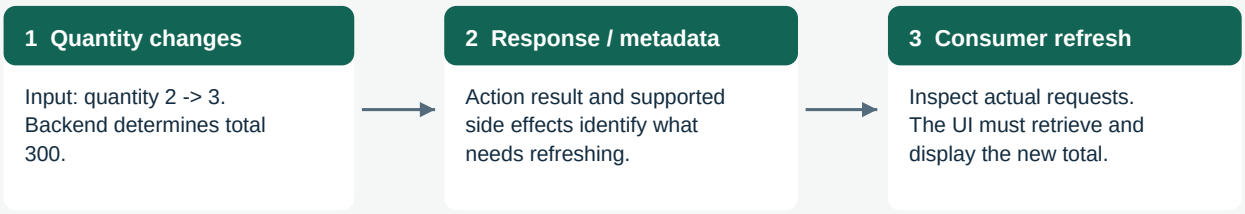
SAP Learning on authorization

SAP RAP test frameworks and boundaries

Side effects and stale UI diagnosis

The backend is right, but the screen is stale

Separate business-state correctness from the consumer refresh contract



Observation	Next investigation
Backend still says 200	Calculation / transactional read
Backend says 300; UI says 200	Side-effect targets / response / binding

A page reload can hide the symptom. It does not explain the missing refresh contract, and another commit is not a refresh mechanism.

Use the observed backend value to choose the next debugging branch.

Explain it: What request proves that the consumer refreshed the calculated total?

Limit: Consumer and metadata support are release-dependent; a valid backend side effect is not proof that every client processes it.

Related questions: ADV-07, SRAP-21, OPS-06

SAP: General RAP BO implementation contract

SAP Learning on authorization

SAP RAP test frameworks and boundaries

RAP message lifecycle and correction

State messages need an invalidation story

Example: an invalid child date is corrected in a later draft request

Moment	Business state	Message decision
Request 1	Child date inconsistent	Report state error for that child
Request 2	User corrects the date	Invalidate relevant prior state message
Revalidation	State is now consistent	Do not recreate the old error
Transition failure	Operation itself failed	Return transition-related feedback

Trace instance identity, state area and element/path targeting. A correct text attached to the wrong child is still incorrect behavior.

REPORTED carries messages; the supported FAILED response rejects instances. Message severity alone is not a general rejection mechanism.

Follow the same child through an error and correction instead of treating a message as timeless text.

Explain it: Which identity and state area would you inspect when the old error remains?

Limit: Message lifetime and permitted state-message scenarios depend on active/draft and the supported operation contract.

Related questions: ADV-08, SRAP-24, RAP-07

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

SAP Learning on authorization

Calculation placement and query semantics

Optimize at the layer that owns the requirement

A remote virtual field changes the query cost and supported operations

Requirement	Possible design	Verification
Display enrichment	Batch remote lookup	Calls and missing-data policy
Filter before paging	Push predicate / persisted value	Complete result semantics
Sort million-row list	Supported sortable data model	Stable global order
Analytical aggregate	Correct grain / pushdown	Units, currency and row count

Bad shortcut: enrich one page, then sort it

Sorting ten fetched rows does not produce the correct top ten across a million candidates. A display calculation does not automatically support database filtering or sorting.

Choose supported provider and consumer capabilities on the target release; do not assume every virtual element can push filters or ordering down.

Use the requirement to decide where a value must exist before filtering, ordering or aggregation.

Explain it: Why can a correct calculation still yield a wrong page of results?

Limit: Architecture alternatives, not a promise that all listed options are supported by each projection/custom provider.

Related questions: CDS-09, MODEL-07, MODEL-08, MODEL-09, MODEL-18, OPS-08

SAP Learning on CDS concepts

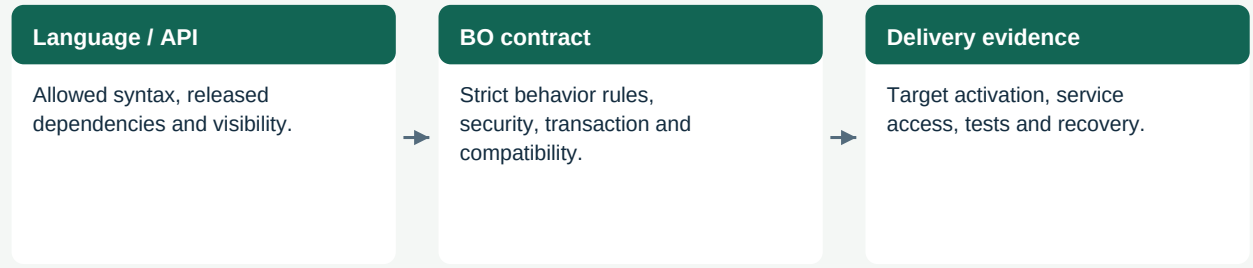
SAP RAP authorization control

SAP RAP test frameworks and boundaries

RAP release and consumption gates

A green language check is one gate, not the whole release

Move from source eligibility to the consumer and operational contract



A released BO interface can expose a narrower supported contract than the underlying implementation. Consume the permitted layer, even when ADT can display more objects.

strict(2), C1 release and a working service binding are different facts. None alone certifies production readiness.

Review eligibility, behavior and delivery as separate evidence gates.

Explain it: Which gate would catch an API-compatible change that breaks deployment configuration?

Limit: A teaching checklist; exact release contracts, component visibility and delivery tools depend on the product.

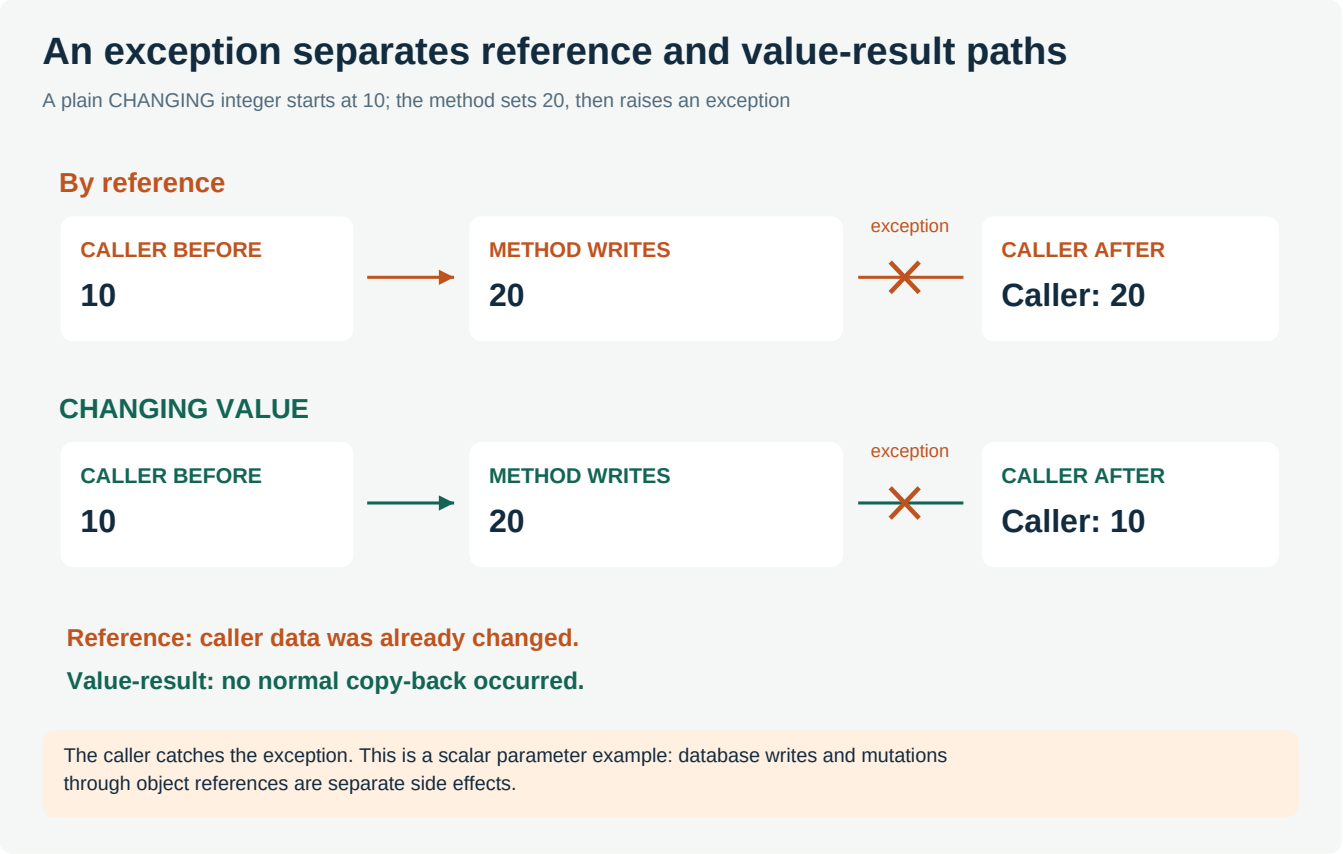
Related questions: SRAP-16, SRAP-22, CLOUD-06, CLOUD-10, CLOUD-13, CLOUD-17, OPS-09, OPS-10, OPS-14

SAP ABAP Cloud development model

SAP: General RAP BO implementation contract

SAP RAP test frameworks and boundaries

Parameter values after exceptional exit



Compare the same assignment under two transfer modes. The final caller value is visible evidence of the difference.

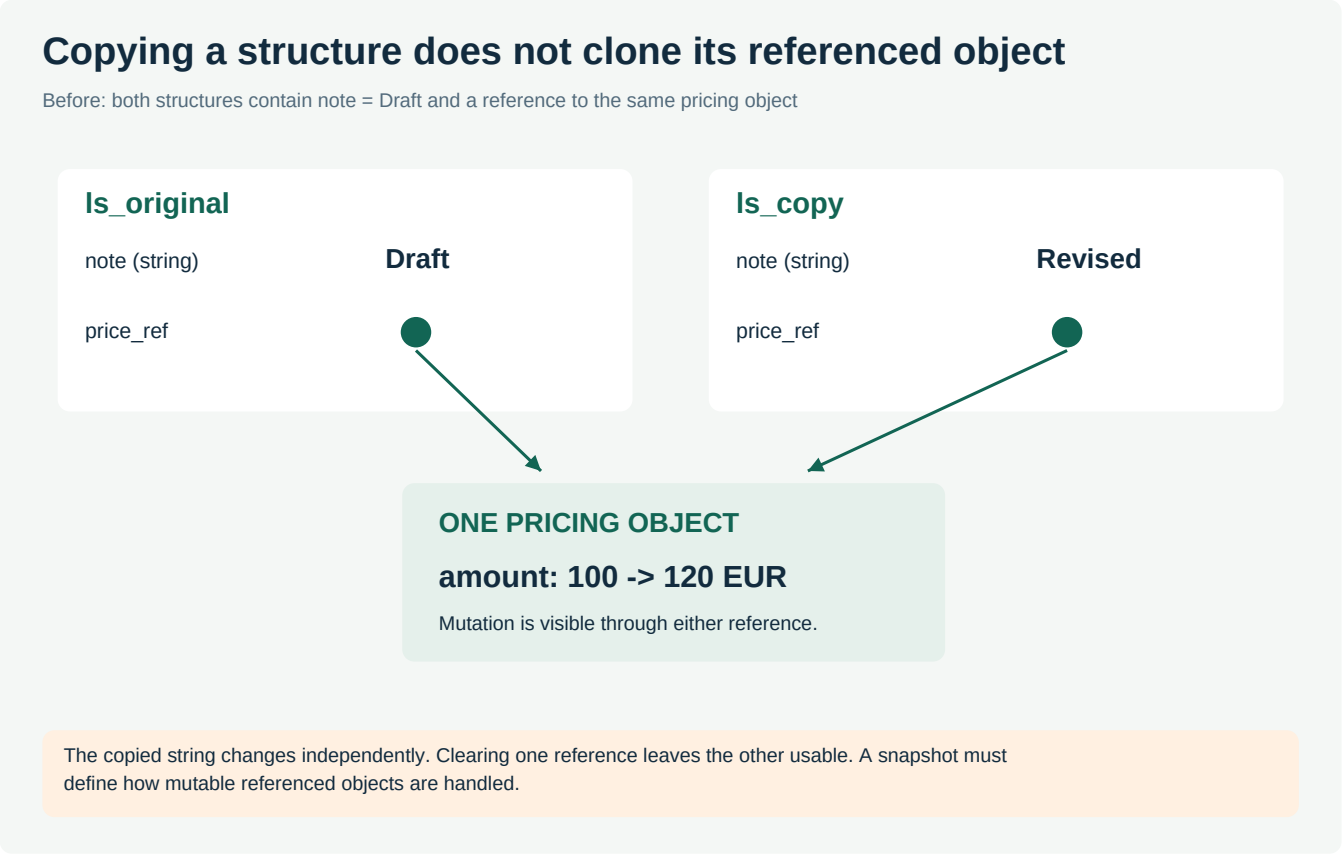
Explain it: Which result changes if the method returns normally? What changes if the parameter holds an object reference?

Limit: This assumes a plain integer, value-result passing, and no additional side effects.

Related questions: BASE-01

SAP: Pass by value

Value components versus a shared referent



Two structure values own independent string values but lead to the same pricing object. The arrow targets explain why a getter can expose mutable state.

Explain it: Which change affects only ls_copy, and which change affects an object visible through both?

Limit: Conceptual object identity, not physical memory addresses or a particular sharing optimization.

Related questions: BASE-02, OODEEP-06, OODEEP-10

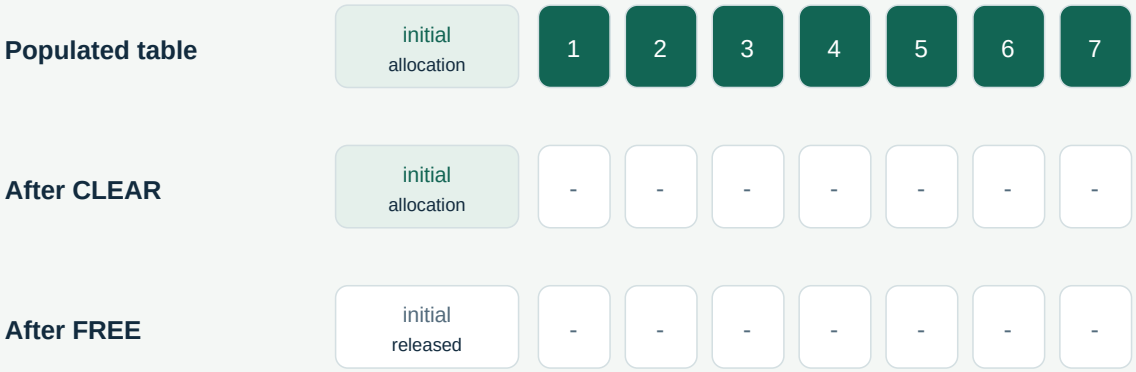
SAP: Pass by value

SAP Learning: Documenting ABAP code

Three distinct state and memory lifetimes

Row contents, allocation, and class state are different

Conceptual memory ledger: bars are not measured sizes



NEW service( ) resets neither a class cache nor another reference.

Instance lifetime != static lifetime != resource ownership.

CLEAR retains the table initial allocation; FREE releases it too. Neither guarantees immediate operating-system memory shrinkage or resets CLASS-DATA.

Empty row slots and the initial allocation are deliberately drawn separately. A new object does not imply a new internal session or empty class cache.

Explain it: Why can recreating a service leave a test-order failure unchanged?

Limit: Allocation diagram is qualitative; measure actual ABAP memory and cache lifetime in the target context.

Related questions: BASE-03, OODEEP-08

SAP: Pass by value

SAP Learning: Documenting ABAP code

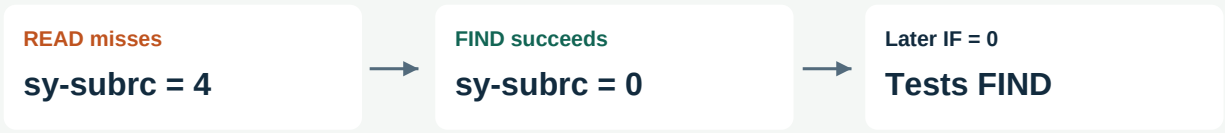
Stale aliases and overwritten return codes

A result belongs to the operation that produced it

Two sequential defects: stale field-symbol target and overwritten return code

Step	Operation	Result after this step
1	ASSIGN component PRICE	<field> -> PRICE; sy-subrc = 0
2	ASSIGN component PRCIE fails	Old assignment may remain; result = 4
3	Write through old alias	PRICE can be changed accidentally

Separate return-code trap



Consume or preserve the latest result immediately. Use ELSE UNASSIGN where supported, or unassign before the dynamic assignment and handle failure.

The first sequence can write to the previous target. The second takes a success branch for a different operation than the programmer intended.

Explain it: Which test requires a successful operation before the failure to expose the defect?

Limit: Only statements documented to set sy-subrc replace it. Assignment failure behavior depends on the exact form and additions.

Related questions: BASE-04, BASE-05

SAP: Pass by value

Conversion, rounding, and reduction counterexamples

Small numeric examples expose three different contracts

Representability, business rounding, and accumulator initialization are separate decisions

LOSSLESS CONVERSION



Half-up to 2 decimals	Calculation	Total
Round each line, then sum	0.01 + 0.01	0.02
Sum 0.005 + 0.005, then round	round(0.010)	0.01

MAX of {-8, -3}: correct result = -3

Starting the accumulator at 0 invents a larger candidate that is not in the input.

Formatting is not validation. Specify rounding stage and mode, and define empty-input semantics before choosing an expression.

A lost character, a one-cent difference, and a false maximum reveal three unrelated errors that concise syntax can hide.

Explain it: Which expected results change with a different rounding mode, and which are independent of rounding?

Limit: Half-up and two decimals are explicit exercise assumptions, not a universal currency rule. MAX notation is explanatory.

Related questions: BASE-06, BASE-07, BASE-11

SAP: Pass by value

Three mapping cases and two presence cases

Preserving a base is not the same as preserving omission

Saved structure: status = Approved; note = Old

Source shape / value	Plain CORRESPONDING	With BASE(existing)
note = New; no status field	status becomes initial	status stays Approved
note = New; status is initial	status becomes initial	status becomes initial
note = New; status = Draft	status becomes Draft	status becomes Draft

Optional parameter presence is another explicit signal

Call	Value	IS INITIAL	IS SUPPLIED
Argument omitted	0	true	false
Argument explicitly zero	0	true	true

A wrapper that always forwards an optional argument makes it supplied downstream. BASE only preserves components not overwritten by matching source components.

The matching initial status overwrites BASE just as a populated status does. A separate parameter table shows why zero cannot encode omission.

Explain it: Which source needs a presence flag when the source type always contains status?

Limit: Structure mapping example; internal-table BASE preserves rows under different construction semantics.

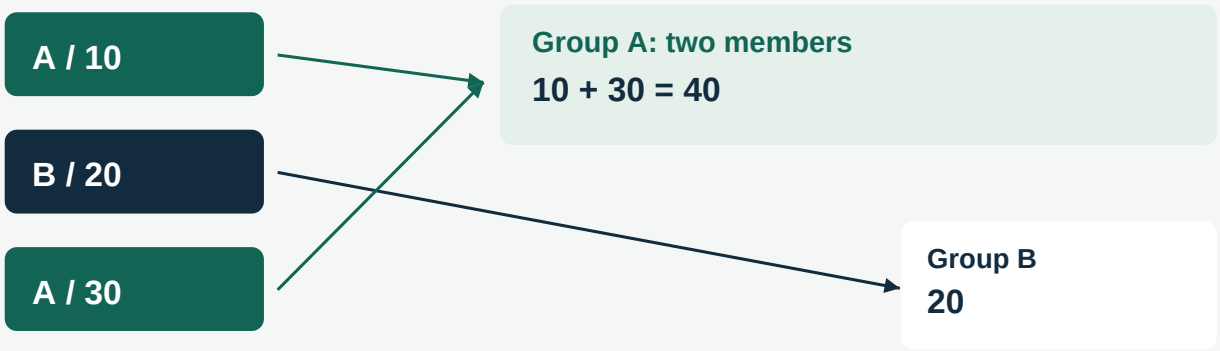
Related questions: BASE-08, BASE-14

SAP: Pass by value

Unsorted group membership and explicit filter keys

Grouping collects members; FILTER chooses a key path

Arrival order: A/10, B/20, A/30. No pre-sort is required for GROUP BY customer.



Standard table + empty primary key: basic FILTER needs more.

A suitable sorted secondary key by customer can be selected with USING KEY.

Group body: once per group; member loop: once per member. FILTER predicates must satisfy the selected sorted/hash key rules.

Arrows bring nonadjacent A rows into one group. The grouping total is computed from members, not read from the representative row.

Explain it: What changes with WITHOUT MEMBERS, and why is an arbitrary OR predicate not automatically a basic FILTER condition?

Limit: Source rows and totals are illustrative; no measured performance advantage is asserted.

Related questions: BASE-09, BASE-10

SAP: Pass by value

Duplicate insertion and empty authorization boundaries

An empty set and an empty selection range are not alike

Two boundary cases that need an explicit business policy

UNIQUE INTERNAL-TABLE INSERT

Key	Saved amount
7	10

Try key 7 / amount 20

Duplicate: not an update.

Existing amount remains 10.

NO PERMITTED PLANTS

range = [ ]

IN initial_range means no restriction.

No authorization must return no rows or reject before the otherwise unrestricted query.

Duplicate rejection does not define a merge policy. Empty authorization is not an optional empty filter; keep the permission decision explicit.

A duplicate unique key preserves the original row for the specified insertion form. An initial selection range has a very different meaning from an empty list of permissions.

Explain it: How would you distinguish identical duplicates from conflicting ones, and prove a user with no plants sees nothing?

Limit: Single-line INSERT INTO TABLE with a unique primary key; other insertion and key-conflict forms need their own documented handling.

Related questions: BASE-15, BASE-16

SAP: Pass by value

Two input kinds and two safe handling paths

Bind values; select identifiers from a controlled vocabulary

Input: customer name = O'Brien; sort option = customer

DATA VALUE

O'Brien

Bind through @customer_name.
An apostrophe remains data.

not SQL
from the client

USER SORT OPTION

customer

Map allowed customer option
to a server-chosen column.

Authorization conditions remain mandatory and independent of user filters.

Quoting a string cannot authorize a column, table, operator, or whole clause. Keep the SQL shape static wherever possible; reject unsupported sort options.

The apostrophe is intentionally harmless data on the left. The sort request on the right selects a predefined identifier rather than supplying executable SQL.

Explain it: Why would quoting the sort option as a literal fail to solve identifier validation?

Limit: Conceptual SQL boundary; available dynamic-programming helpers and API release status depend on the target environment.

Related questions: BASE-12

SAP: Pass by value

Exceptional exits and preserved causes

CLEANUP is unwinding work, not a universal finally

An application-owned resource has an explicit release obligation

Exit path	CLEANUP in this TRY?	Required design
Normal completion	No	Explicit normal release
Handled by same TRY	Not merely for this catch	Cover this handled path
Propagates to outer handler	Relevant unwinding path	Restore local consistency

Translate meaning; retain the diagnostic cause

previous

TECHNICAL EXCEPTION

Connection unavailable

→

SERVICE EXCEPTION

Price service unavailable

An empty price is not a safe default for a failed lookup. Release the resource you own, preserve the cause, and do not hide technical failure as business absence.

The table names three exit paths that require separate reasoning. The exception chain carries the lower-level diagnosis through a meaningful service-level failure.

Explain it: Which path would a test of only propagated exceptions miss?

Limit: Simplified non-resumable scenarios; exact nesting and handlers determine CLEANUP execution. Resource ownership remains application-specific.

Related questions: BASE-13, OODEEP-09

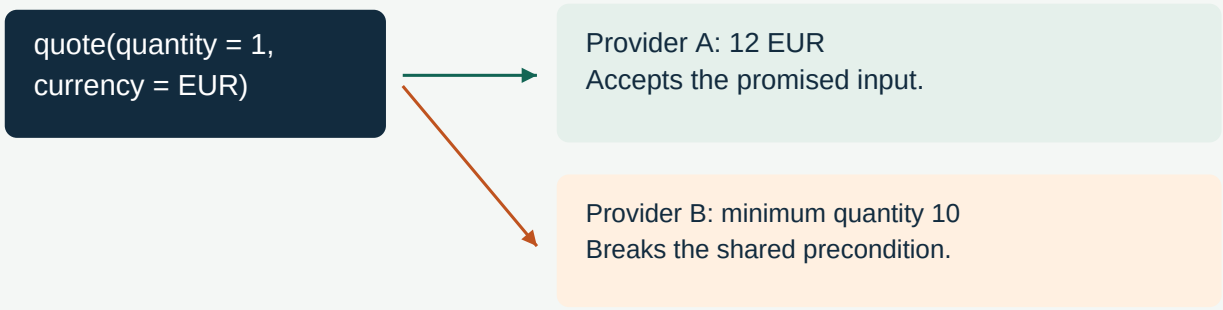
SAP: Pass by value

SAP Learning: Documenting ABAP code

A concrete counterexample to substitutability

Same call shape does not prove the same behavior

Shared quote contract: accept every positive quantity; return the requested currency



Change request	Contract-preserving direction
Different inherited implementation	REDEFINITION keeps the signature
Optional simulation capability	Separate capability; honest unsupported result

The interface supports common dispatch across unrelated classes. It does not enforce business substitutability. Shared code alone does not justify a superclass.

A valid quantity-one call exposes a behavioral violation invisible in matching parameter types. The lower table separates redefinition from a genuinely new capability.

Explain it: Would downcasting every caller to Provider B repair the contract? What else besides accepted inputs must agree?

Limit: Hypothetical contract; a narrower published domain would lead to a different assessment. No executable interface is implied.

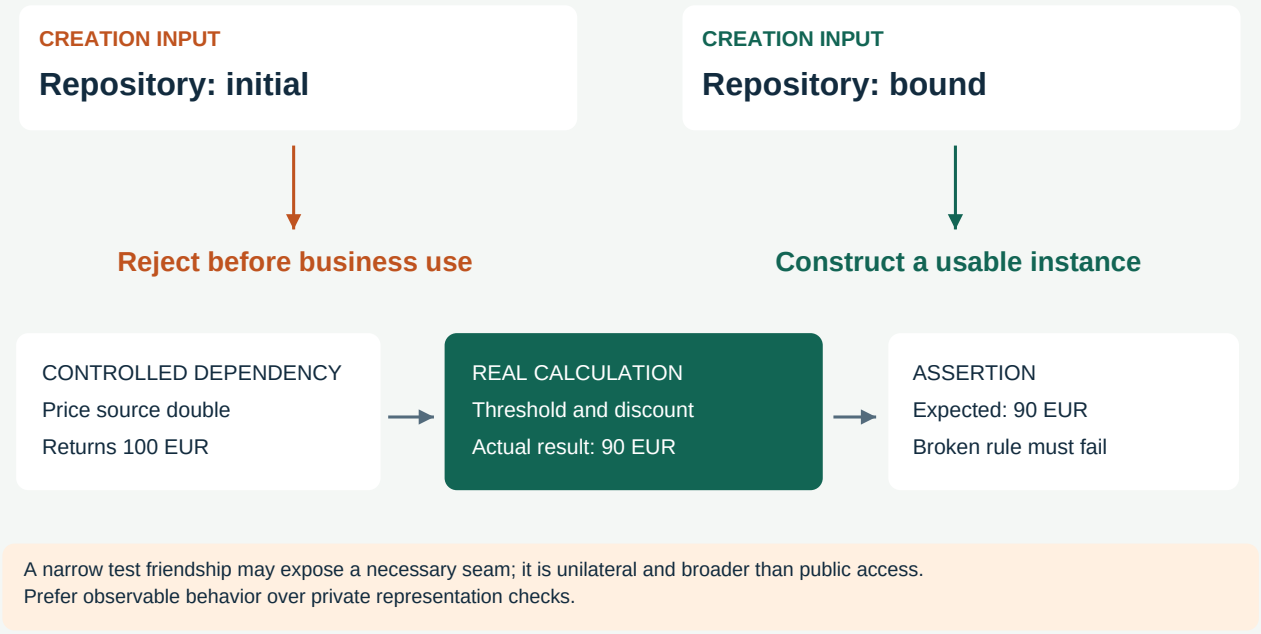
Related questions: OODEEP-01, OODEEP-02, OODEEP-03, OODEEP-04, OODEEP-11

SAP Learning: Documenting ABAP code

Construction invariants and meaningful test boundaries

Build a valid object, then test the real decision

Approval service: a repository is mandatory; the discount threshold remains real code



A narrow test friendship may expose a necessary seam; it is unilateral and broader than public access.
Prefer observable behavior over private representation checks.

Invalid dependencies are rejected before use. In the successful test, the input price is controlled but the discount rule itself executes.

Explain it: What defect should make the assertion fail, and which private access is actually necessary?

Limit: The 10-percent discount is an illustrative fixture, not an application rule. Friendship details and real integration require separate checks.

Related questions: OODEEP-05, OODEEP-07, OODEEP-12

SAP Learning: Documenting ABAP code

Strategy versus Template Method in approval rules

One variation: replace a rule or specialize a step?

Worked choice: approval policy varies by country; process ownership must stay stable

STRATEGY / composition

Approval coordinator -> rule interface

Country A rule
Limit: 500 EUR

Country B rule
Limit: 900 EUR

700 EUR: A rejects; B accepts under this fixture.

TEMPLATE METHOD / inheritance

1. **Read inputs**
2. **Calculate policy**
3. **Return decision**

Stable skeleton; subclass supplies the chosen step.

Assembly decision	Boundary to keep
Factory selects policy	Validate country and policy version
Constructor receives rule	Rule returns a decision; does not commit

The amounts are fictional policy fixtures. Choose a real variation point; a reusable calculation alone does not require an inheritance hierarchy.

The same 700 EUR request receives different policy decisions through the same Strategy contract. Template Method instead fixes a call skeleton and exposes deliberate subclass steps.

Explain it: Which part varies, who selects it, and which object keeps transaction ownership?

Limit: Conceptual pattern comparison, not executable ABAP or a universal approval rule. Framework-owned construction may require a supported delegation seam.

Related questions: PAT-01, PAT-10, PAT-12

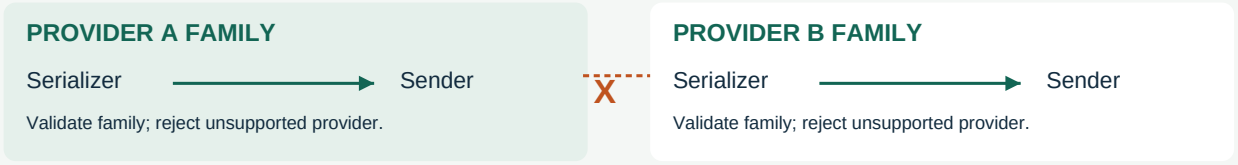
SAP ABAP pattern examples: Strategy

Three factory choices and a concrete family constraint

Construction must protect a real invariant

Worked family: provider A serializer must be paired with provider A sender

Approach	Construction decision	Use when
Simple factory	Explicit supported choice	Small, stable implementation set
Factory Method	Creator subclass supplies object	Creator extension is intentional
Abstract Factory	Factory supplies related family	Products must remain compatible



A factory is useful here because an A/B mixed pair is invalid.

A static method returning an object is not automatically Factory Method. Do not create a product-family abstraction when there is only one trivial product.

The family boundary prevents an A serializer from being combined accidentally with a B sender. The table distinguishes a simple choice from subclass creation and related-product creation.

Explain it: What invalid combination does this factory prevent, and when would a simple factory be enough?

Limit: Provider compatibility is a fictional fixture. Pattern names do not grant API release eligibility, security or transaction guarantees.

Related questions: PAT-02, PAT-11, PAT-12

SAP ABAP pattern examples: Strategy

Adapter, Facade and Decorator with observable outcomes

Different wrappers solve different problems

Worked posting boundary: translate data, simplify steps, observe execution

ADAPTER	FACADE	DECORATOR
Domain input: amount = 0, currency = EUR Legacy mapping: value + change indicators	One business operation: post purchase request Coordinates: prepare -> check -> post	Same service interface: timing around the call Preserves: result + failure contract

Worked ordering: one logical call, two attempts

Outer log: 1 operation → Retry: 2 attempts → Eligible read client

Move logging inside Retry -> observe 2 attempts instead of 1 logical call.

Retry is not safe for every write. These patterns cannot remove nested commits, authorize an operation, or release an internal SAP dependency.

Mapping, orchestration and added behavior are different responsibilities. The lower example shows why wrapper order changes whether a log records one logical call or two attempts.

Explain it: Where would you translate an explicit clear, stop after a failed check, and record retry attempts?

Limit: The retry example is deliberately an eligible read. Write retries need the actual idempotency and uncertainty contract; no pattern automatically supplies it.

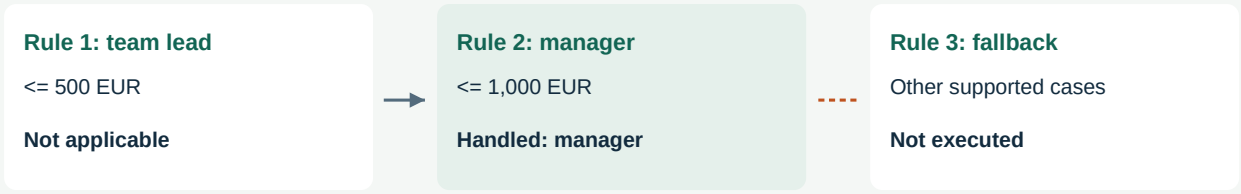
Related questions: PAT-03, PAT-04

SAP ABAP pattern examples: Strategy

First-match routing and idempotent command execution

Route once; execute with a replay contract

Worked request: 700 EUR. The ordered rules choose the first applicable approver.



A handler failure is not the same as "not applicable".

Command identity	Delivery	Effective result
SUBMIT-42	First execution	One state transition recorded
SUBMIT-42	Retry after lost response	Defined replay result; no second transition

Command packages the operation. Durable identity makes safe replay possible.

First-match routing differs from execute-all validation. The Command pattern alone supplies neither persistent queuing nor idempotency nor an undo operation.

The 700 EUR fixture passes the team-lead rule and stops at the manager rule. The selected operation then needs its own durable identity to handle duplicate delivery safely.

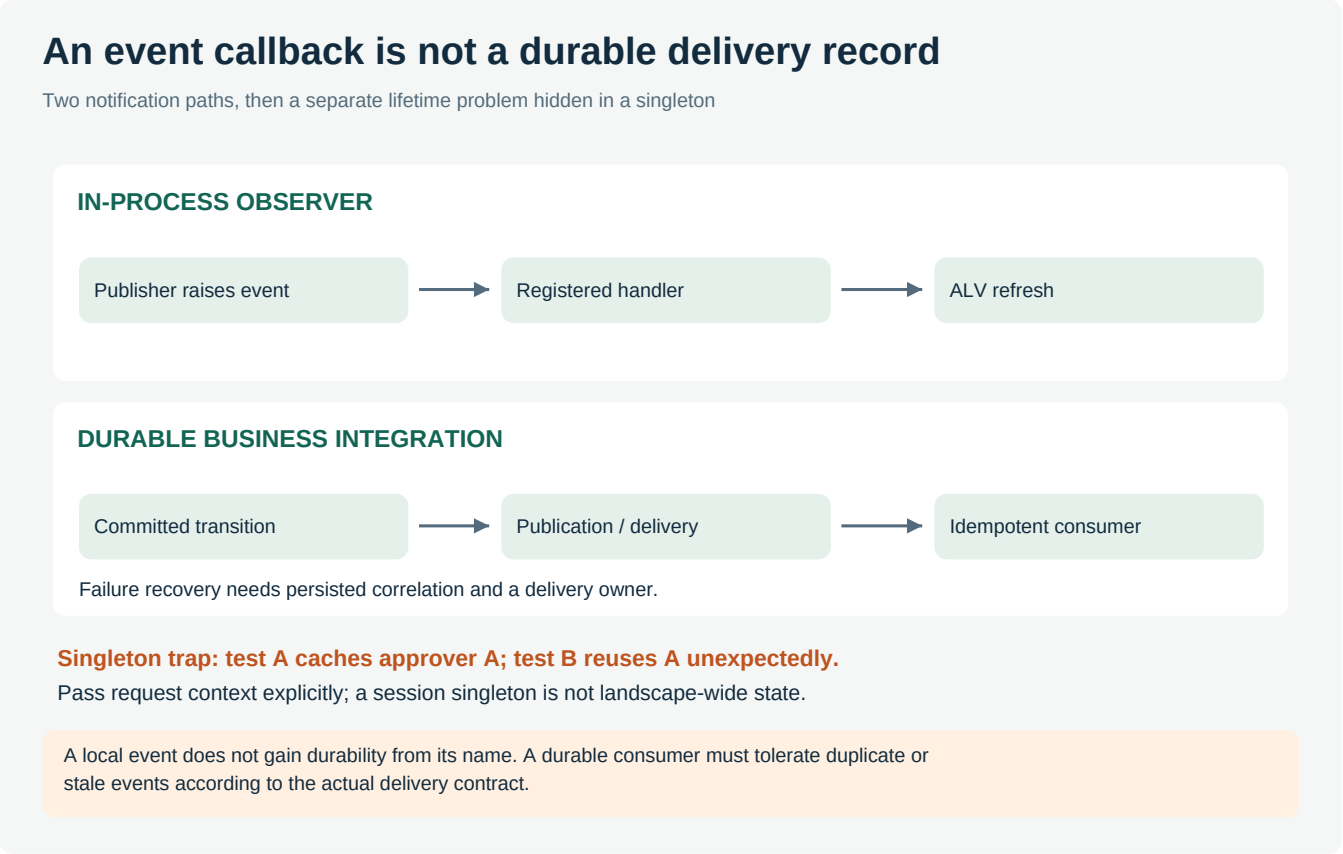
Explain it: What changes if all rules must run, and what must a repeated SUBMIT-42 return?

Limit: Routing thresholds and command identity are fictional. The actual business transition, persistence and compensation contracts must be designed separately.

Related questions: PAT-05, PAT-07

SAP ABAP pattern examples: Strategy

Observer delivery and hidden singleton state



A registered local handler can update an ALV, while an external process needs transactional publication and durable processing evidence. Session-shared mutable state is a separate source of repeated-test failures.

Explain it: Which facts survive a process restart, and which request-specific values must not be cached implicitly?

Limit: The lower path describes required responsibilities, not an automatic feature of ABAP Objects events. Verify the chosen event infrastructure and service lifetime.

Related questions: PAT-06, PAT-09

SAP ABAP pattern examples: Strategy

Repository read semantics and proportional abstraction

A useful seam must preserve the right state

Worked RAP fixture: Order 4711 changes from 100 EUR to 120 EUR before save

Persistence read	Amount		Transactional BO read	Amount
Order 4711	100 EUR	→	Order 4711	120 EUR

Repository contract: which view does this caller require?

Inside a rule after update: the current transactional view may be required.

Persisted reporting read: a different explicit contract may be appropriate.

A double returning 120 does not prove the real adapter reads the buffer.

Design force	Smallest useful response
External dependency blocks a test	Introduce a meaningful seam
One pure, stable calculation	A focused method may be sufficient

Do not bypass managed RAP with repository table writes. Pattern count is not a quality metric; verify unsaved reads, rollback and production wiring.

A direct persistence read returns 100 while the transaction already sees 120. The repository seam is valuable only if its real implementation honors the state contract required by the caller.

Explain it: Which integration test catches a mockable repository that reads the wrong state, and when is no new abstraction needed?

Limit: Conceptual read semantics rather than an SQL isolation guarantee. The specific BO, EML operation, authorization and target release remain authoritative.

Related questions: PAT-08, PAT-10, PAT-11

SAP ABAP pattern examples: Strategy

HANA pushdown: result grain and table-function review

Correct grain before faster execution

Fictional Order 4711: two items; each accidentally joins to two language texts

Item	Amount EUR
10	40
20	60

Correct total: 100 EUR

join →

Item	Language	Amount EUR
10	EN	40
10	DE	40
20	EN	60
20	DE	60

Wrong total: 200 EUR

Review gates before comparing SQL/CDS and AMDP

1 Restore result grain, client isolation and business authorization.
2 For a CDS table function, verify read-only and target-specific client safety.
3 Measure database work, returned rows and end-to-end time on equal results.

One million detail rows -> grouped results can reduce transfer; measure the actual plan.

A faster wrong aggregate is still wrong. General AMDP capabilities differ from the CDS table-function contract; eligible APIs and release support still apply.

A two-language text join duplicates both item amounts. Repair the relationship before accepting a performance rewrite, then verify security, client isolation and the chosen implementation contract.

Explain it: Why would DISTINCT or dividing by two be an unreliable repair, and what metrics would justify pushdown?

Limit: Illustrative data, not a benchmark. Read-only here refers to the cited CDS table-function scenario, not every AMDP method; check target rules and consumption-path security.

Related questions: SQLX-01, SQLX-02

SAP CDS table functions: SQLScript, read-only and client safety

Source transfer, delivery and verified runtime evidence

A source revision is not a deployed runtime

Worked release evidence: connect commit REV-42 to delivery D-17 and target system TST

Evidence layer	What it establishes	What it does not prove
Git commit REV-42	Reviewable source revision	Target activation or import
abapGit source transfer	Supported objects transferred	Complete productive delivery
Target checks for REV-42	Recorded syntax / ATC / tests	Every external integration
Delivery D-17 into TST	Recorded import and revision	Service works until tested

CHOOSE THE TARGET ROUTE
Product, software component, transport or delivery package.

→

VERIFY PREREQUISITES
Dependency versions, import order, service and communication setup.

→

OBSERVE THE TARGET
Revision readback, smoke test, denied-user check, recovery owner.

Local lint and mock tests are useful evidence with a narrower scope.

abapGit and gCTS are different tools. BTP ABAP environment delivery choices must not be imposed on every ABAP Cloud product or landscape.

Four evidence layers answer different questions. A release links the reviewed and tested revision to an actual imported state, then verifies the target consumer and configuration.

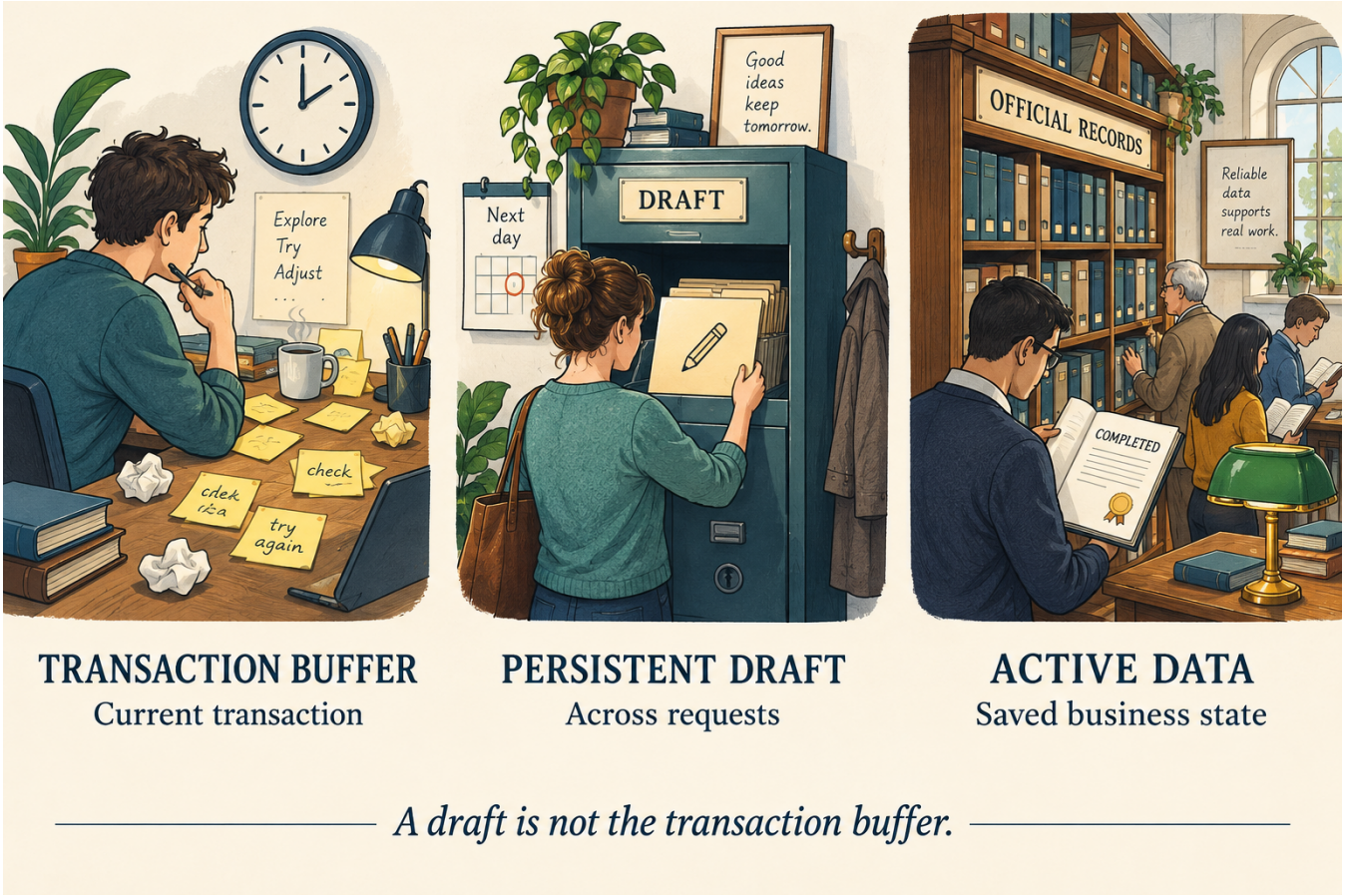
Explain it: Which artifact proves what was imported, and what can fail after a successful import?

Limit: REV-42, D-17 and TST are fictional identifiers. The table is an evidence checklist, not a mandatory sequence requiring abapGit in every transport landscape.

Related questions: DELIV-01

SAP CDS table functions: SQLScript, read-only and client safety

Three storage lifetimes: an illustrated analogy



The desk represents the current transactional buffer; the draft cabinet represents persistent work across requests; the shared archive represents saved active data.

Explain it: Which of the three can survive several HTTP requests without becoming active business data?

Limit: An analogy, not literal SAP storage architecture or a mandatory three-stage pipeline. A new draft may have no active instance. Active business records may still be updated.

Related questions: RAP-03, RAP-12, SRAP-09, OPS-04

SAP Learning on behavior implementation options

SAP: General RAP BO implementation contract

SAP RAP test frameworks and boundaries

Two editors and one version: an illustrated story



Alice and Ben read version 7. Alice updates successfully, producing version 8. Ben must not overwrite that newer state using the stale version 7 precondition.

Explain it: Why should the client refresh and resolve the conflict instead of blindly retrying its old payload?

Limit: Version numbers are illustrative. Real ETags are opaque validators; a failed If-Match precondition commonly returns HTTP 412. This is optimistic concurrency, not a pessimistic lock.

Related questions: ADV-01, ODATA-06, TEST-07, OPS-05, SRAP-10

SAP: General RAP BO implementation contract

SAP Learning on authorization

SAP RAP test frameworks and boundaries